Computer Science
11. HTTPS & REST

11주차: HTTPS, REST

목차 INDEX

01 URL & DNS

- URL
- Domain
- DNS 계층 구조
- DNS Resolution

02 HTTP

- Request
- Response
- HTTP
- Header / Body
- Status Code

03 State & Security

- Stateless
- Cookie
- Session
- HTTP / HTTPS
- Encryption
- MAC/AEAD
- Certificate
- TLS

04 API

- API
- REST
- RESTful

01
URL & DNS

Network 102

URI: Uniform Resource Identifier

URI

https://www.google.com

구글 URL

mailto:helloworld@gmail.com

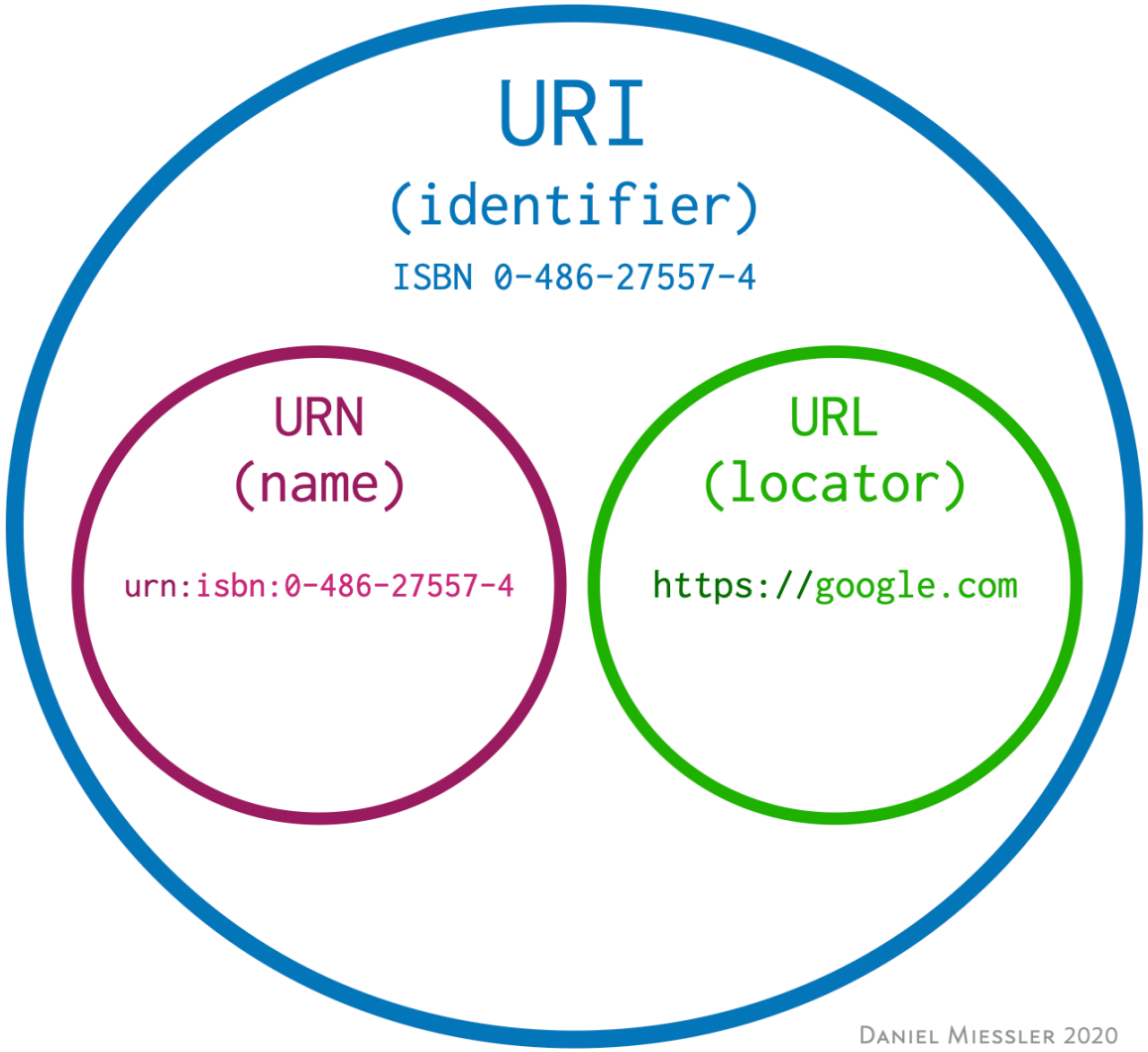
메일 작성

urn:isbn:9780131103627

도서 식별 URN

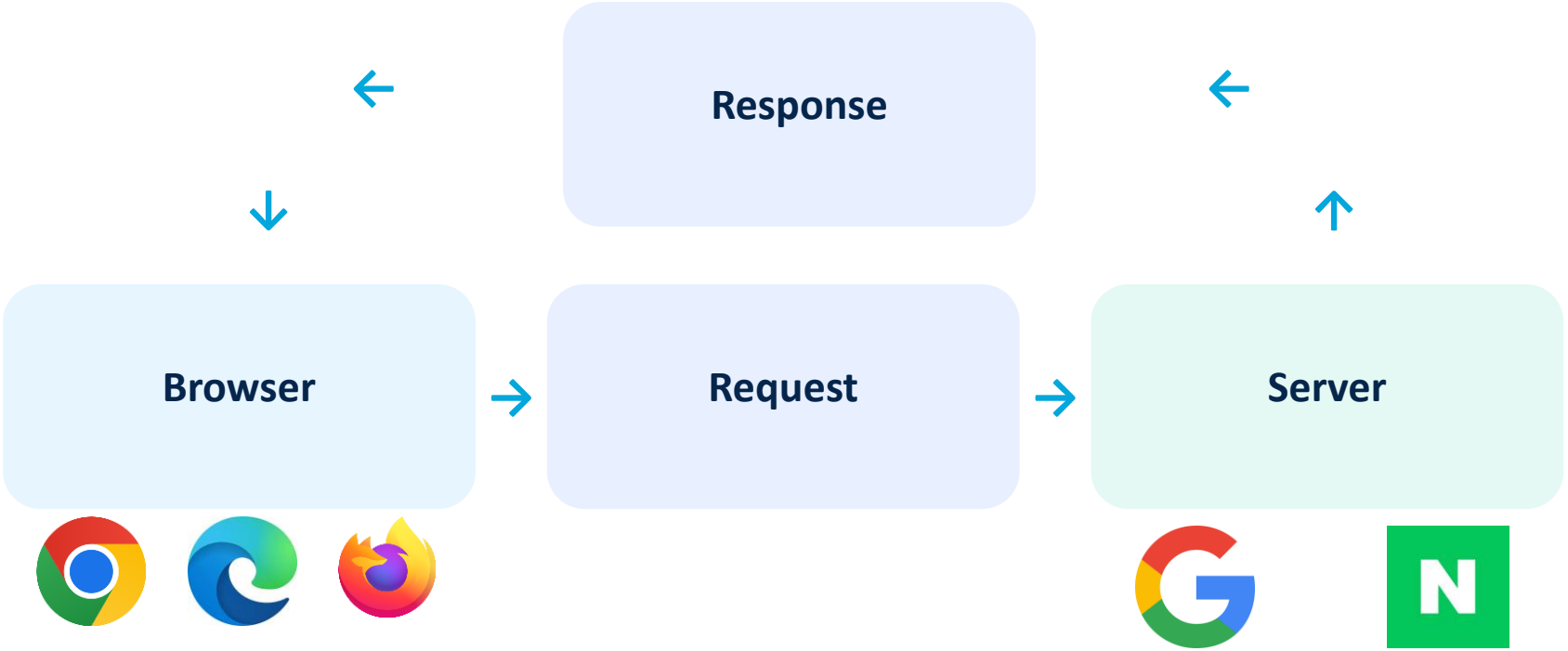
인터넷에서 특정 자원을 식별하기 위한 문자열

URL: Uniform Resource Locator



Daniel Miessler 2020

URL의 구조



URL의 구조


https://www.google.com:443/search?q=url
Scheme
Host
Port
(HTTPS 기본 포트)
Path
Query

URL의 구조

Scheme	통신 방식 및 웹사이트에 접근하는 방법을 지정
Host	접속할 서버의 도메인 이름 (또는 IP 주소)
Port	서버에서 통신할 서비스의 포트 번호
Path	서버에서 요청할 자원의 경로
Query	서버에 전달하는 추가 요청 정보

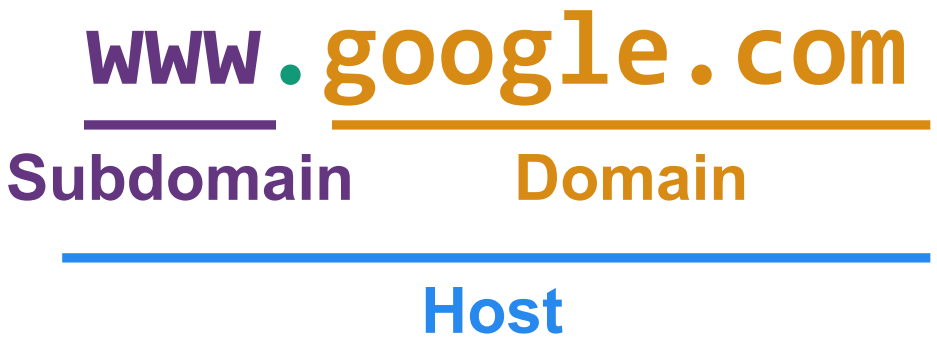
여기서 자원은 인터넷에서 접근하거나 사용할 수 있는 데이터 또는 서비스를 의미

Domain

`www.google.com`

인터넷에서 사용하는 계층적인 이름 체계에 따른 주소
(Host ≠ Domain. Host에는 IP 주소도 사용 가능)

Domain의 구조



Domain은 계층적인 구조를 가짐

DNS: Domain Name System

www.google.com

Subdomain Domain

Host

DNS



Domain의 이름을 IP 주소와 연결하는 시스템

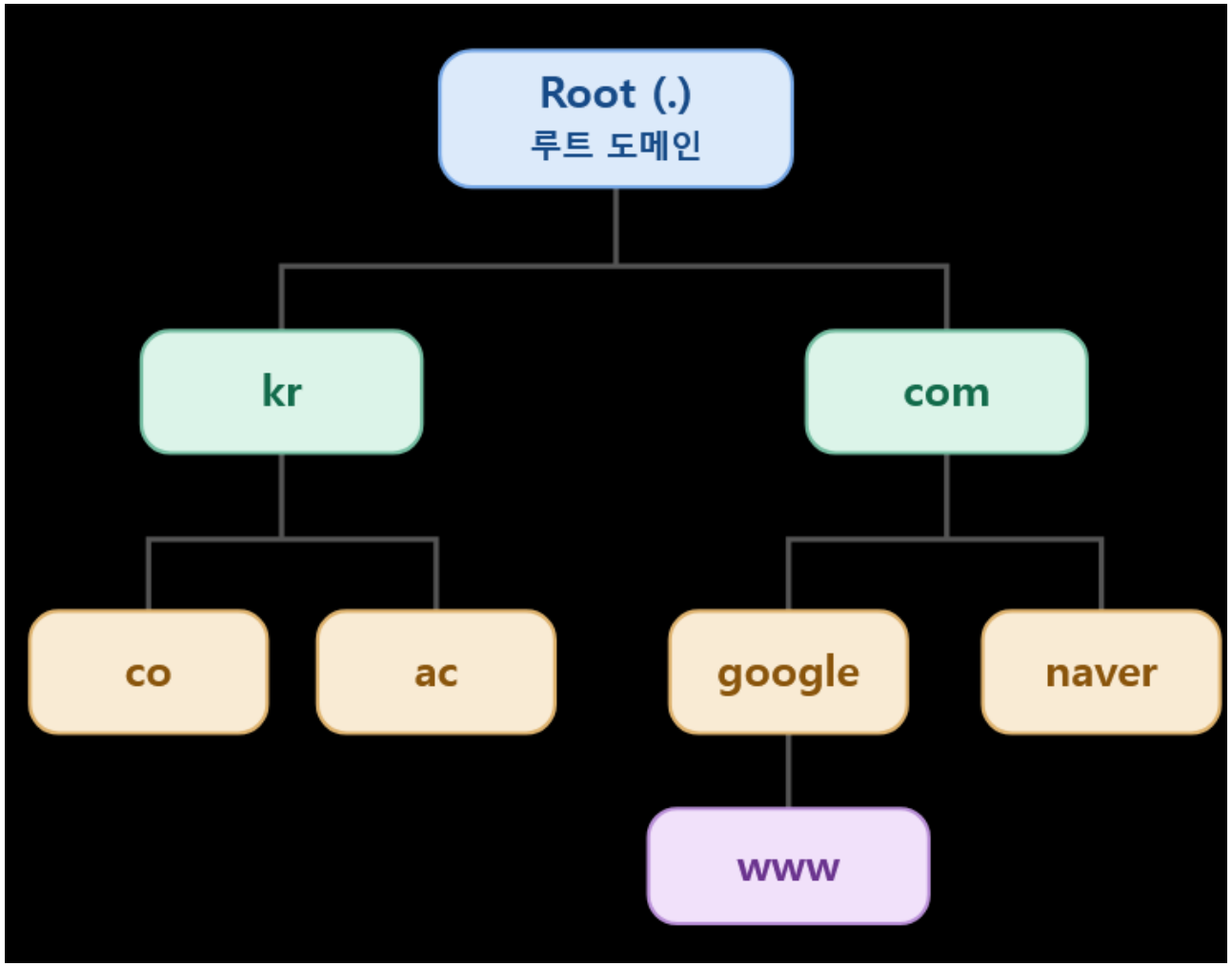
DNS 계층 구조

Root Domain

Top-Level Domain

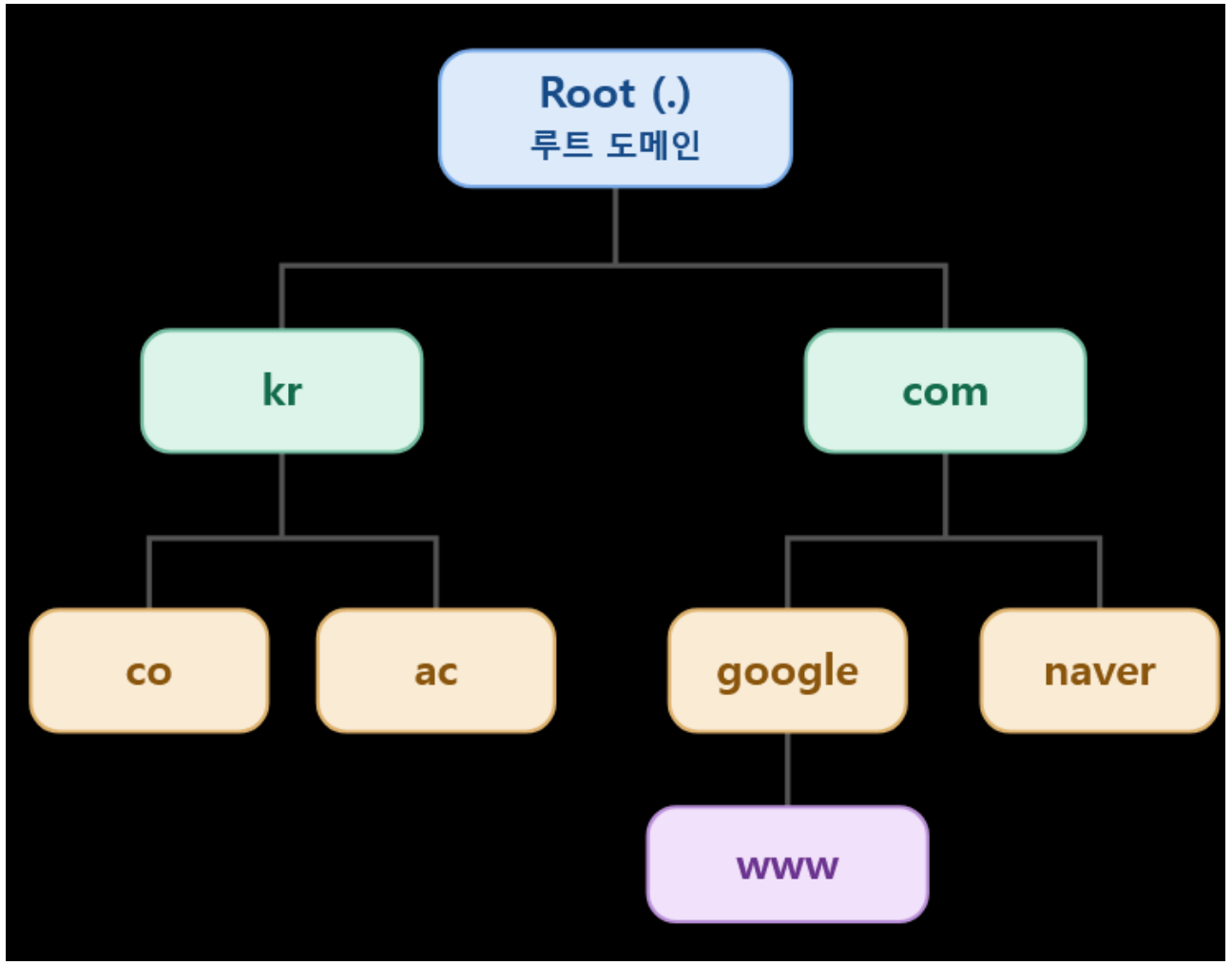
Second-Level Domain

Subdomain

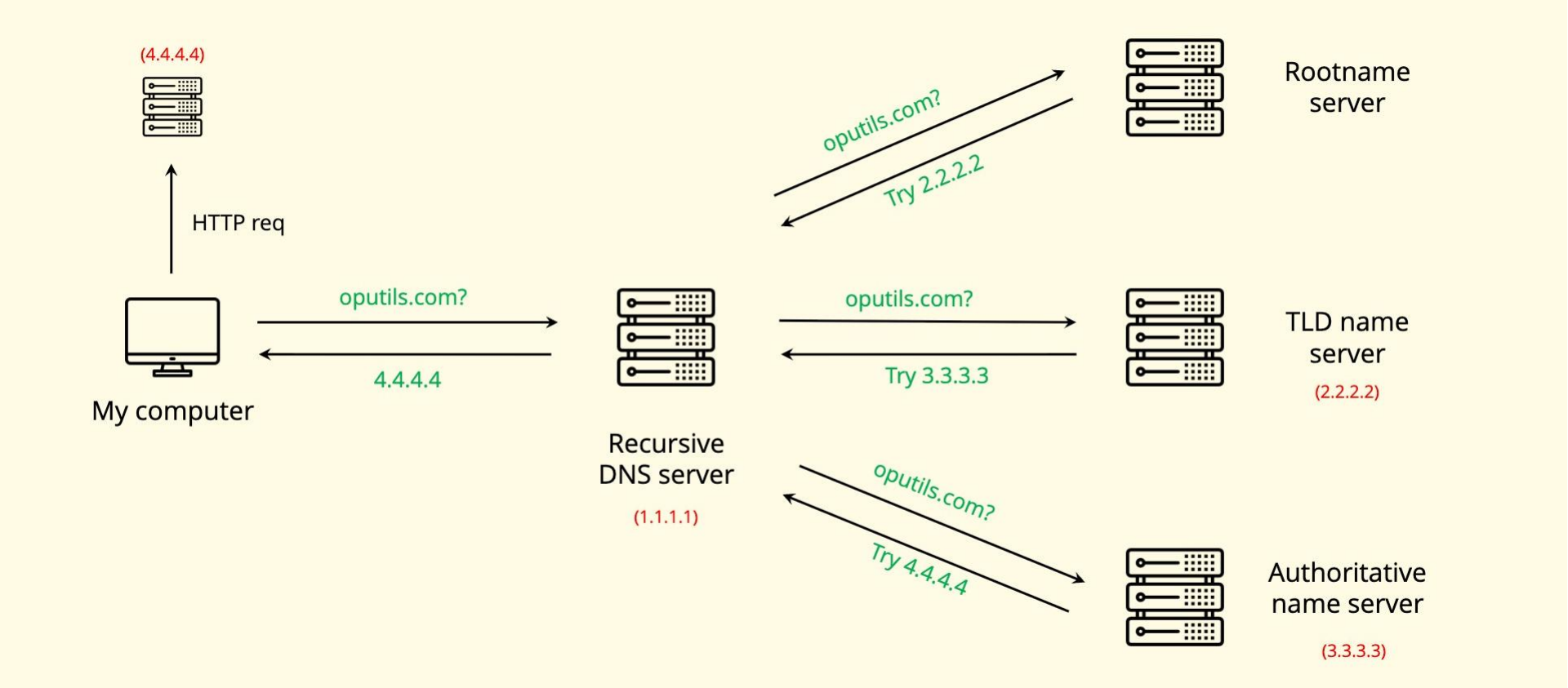


DNS 계층 구조

- 인터넷의 도메인 이름을 체계적으로 관리하기 위한 트리 형태의 구조
- Root: DNS 계층의 최상위를 의미
- Top-Level Domain: 도메인의 최상위 분류
- Second-Level Domain: TLD 바로 아래에 위치하는 도메인
- Subdomain: 상위 도메인 아래에 생성되는 하위 도메인



DNS Resolution



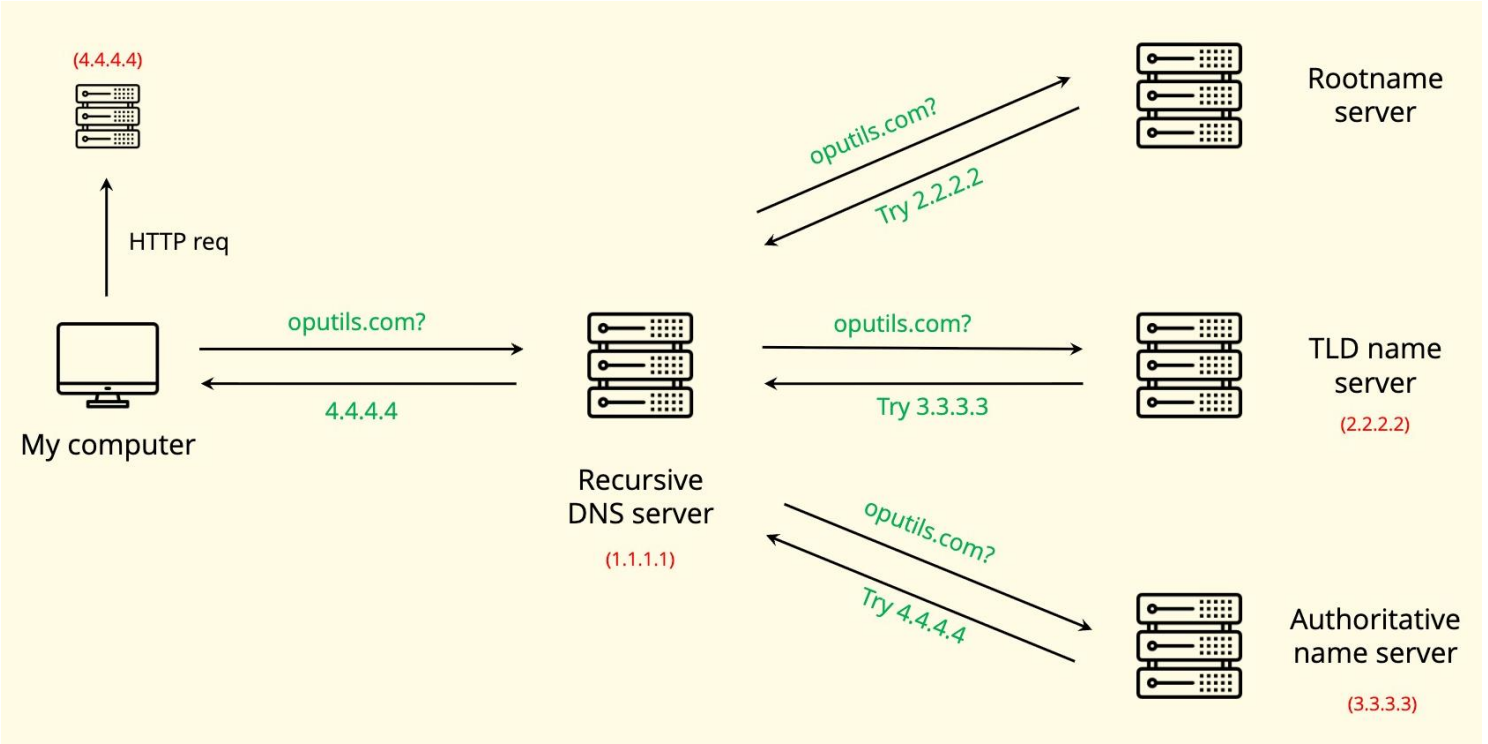
Domain의 이름을 IP 주소로 변환하는 과정

DNS Resolution

- 목표: oputils.com의 IP 주소 알아내기

- 1. 사용자의 컴퓨터에서 DNS Resolver에게 oputils.com의 IP 주소를 알려달라 요청

- 2. DNS Resolver는 DNS 캐시 검색, 만약 정보가 없다면 다른 DNS 서버에게 질문을 시작함



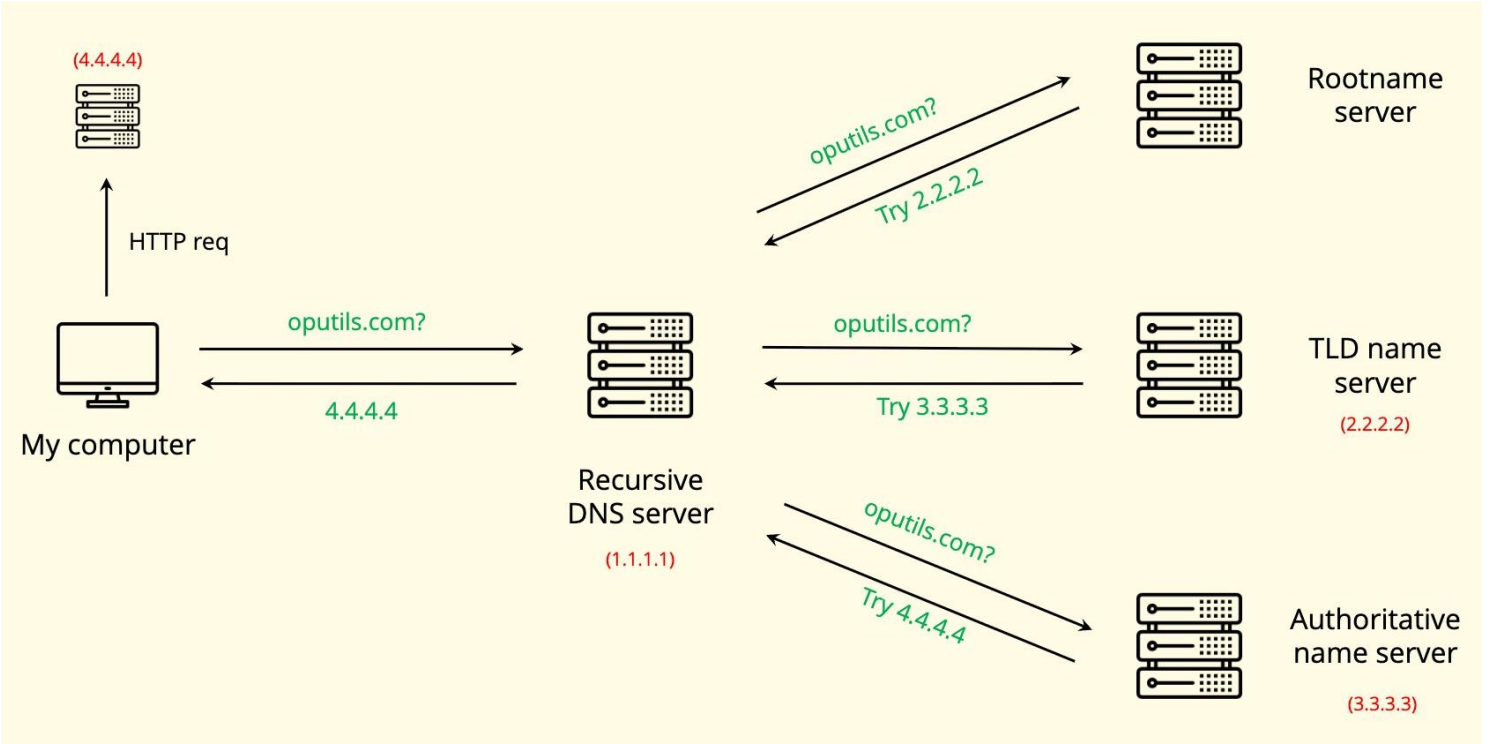
DNS Resolution

- 3. 가장 먼저 Root DNS 서버에 oputils.com의 IP 주소를 물어봄

  ⇒ 루트 서버는 직접 IP를 알려주지 않고 Try 2.2.2.2 라고 응답 (.com을 담당하는 TLD DNS 서버에 물어보라는 뜻)

- 4. .com을 담당하는 TLD DNS 서버에 다시 oputils.com의 IP 주소를 물어봄

  ⇒ TLD 서버는 직접 IP를 알려주지 않고 Try 3.3.3.3 이라고 응답 (oputils.com을 담당하고 권한 있는 DNS 서버에 물어보라는 뜻)

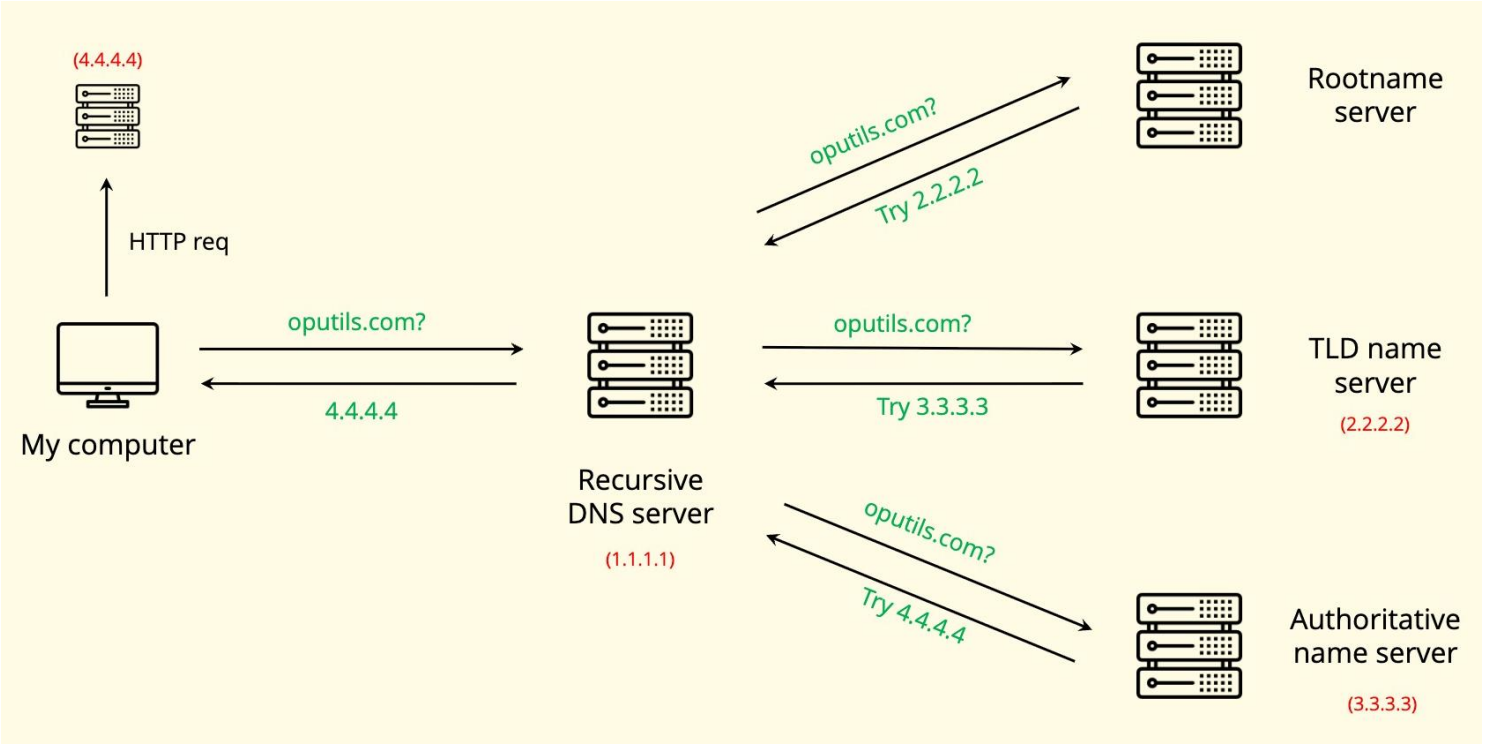


DNS Resolution

- 5. 권한이 있는 DNS 서버에 oputils.com의 IP 주소를 물어봄

  ⇒ 서버는 Try 4.4.4.4라고 응답함 (oputils.com의 IP 주소가 4.4.4.4라는 뜻임)

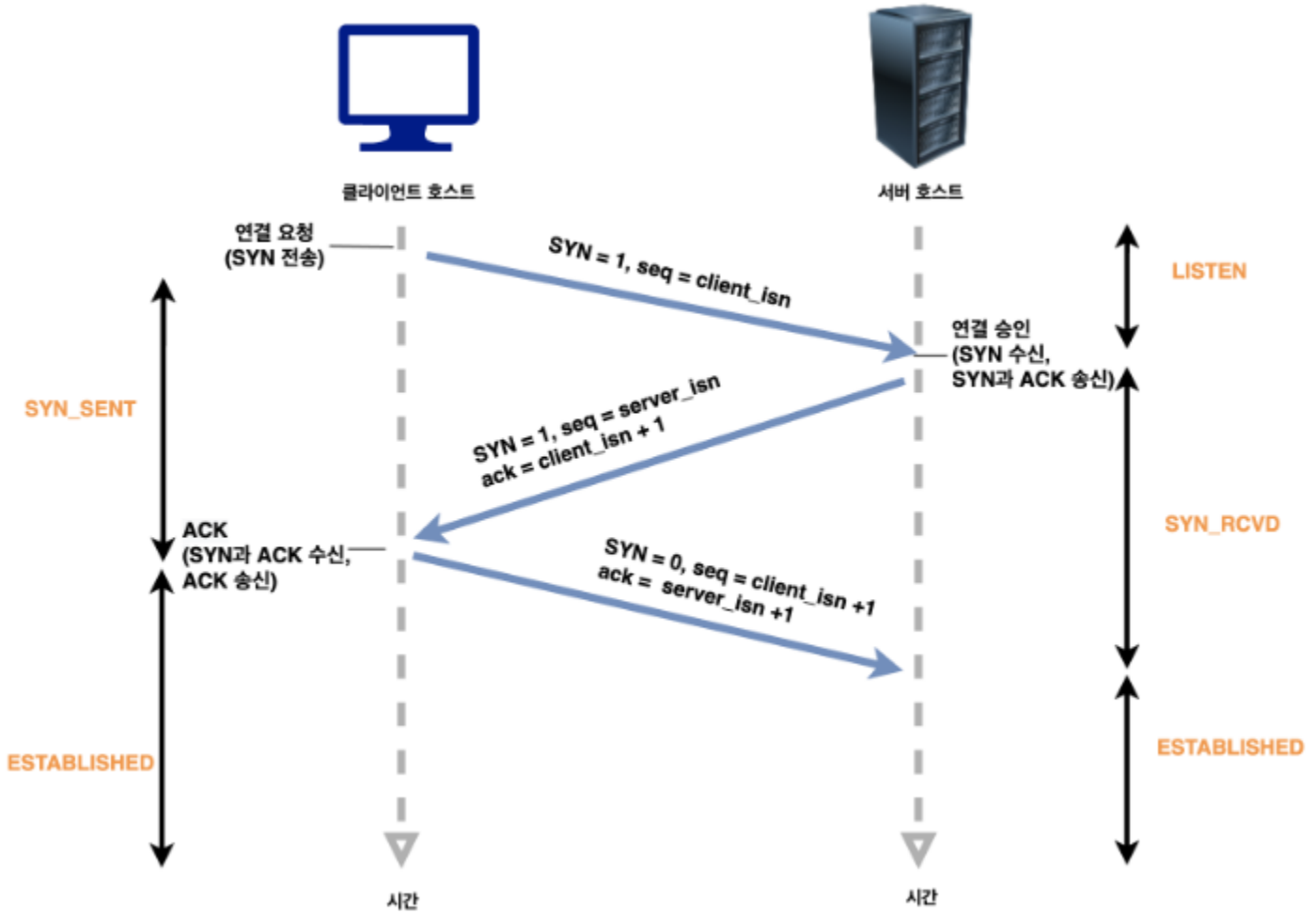
- 6. DNS Resolver가 IP 주소를 사용자 컴퓨터에 반환하고, 컴퓨터는 알아낸 IP 주소를 이용해 웹 서버에 HTTP 요청 전송



02
HTTP

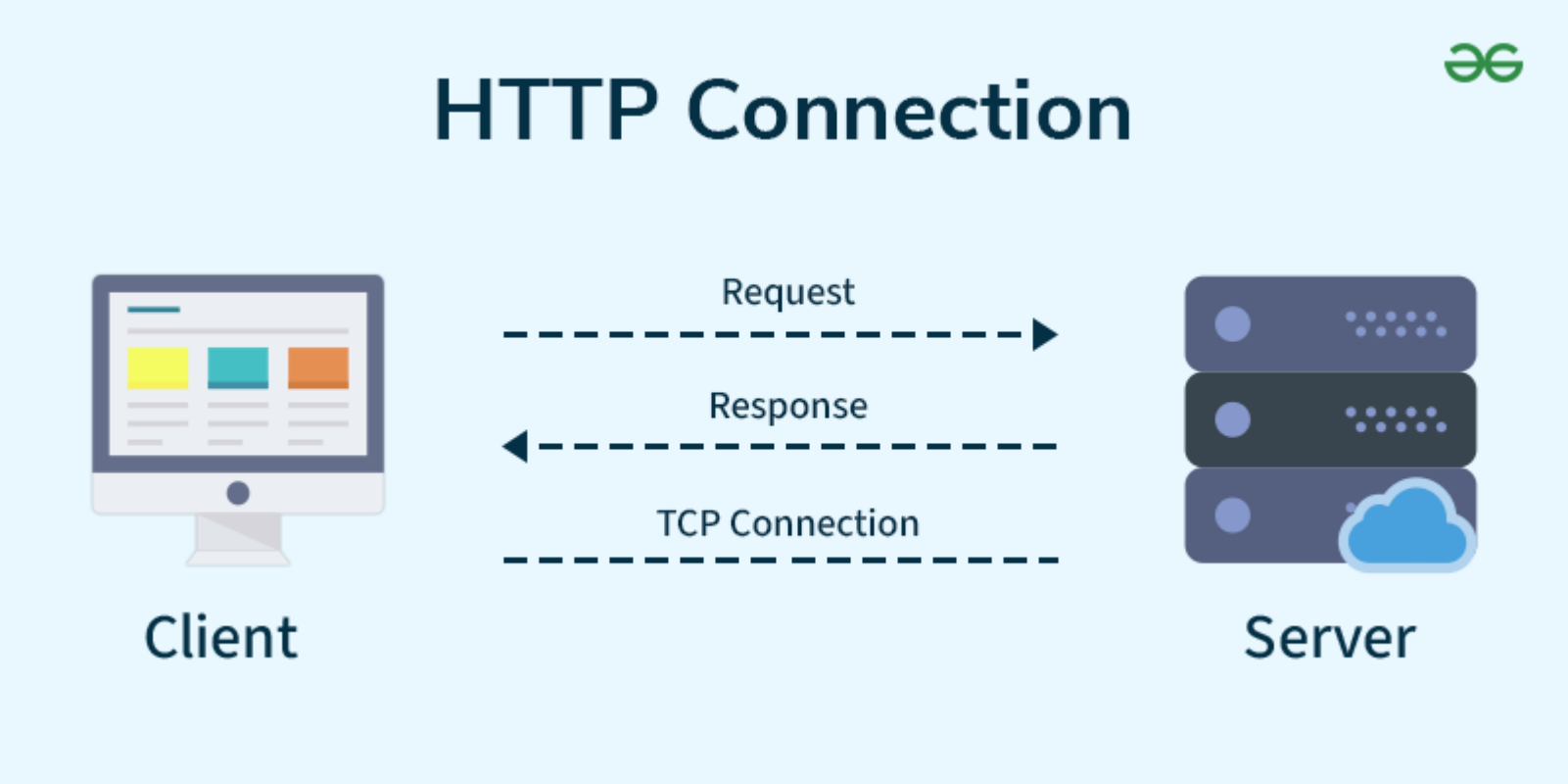
Hypertext Transfer Protocol

TCP: Transmission Control Protocol



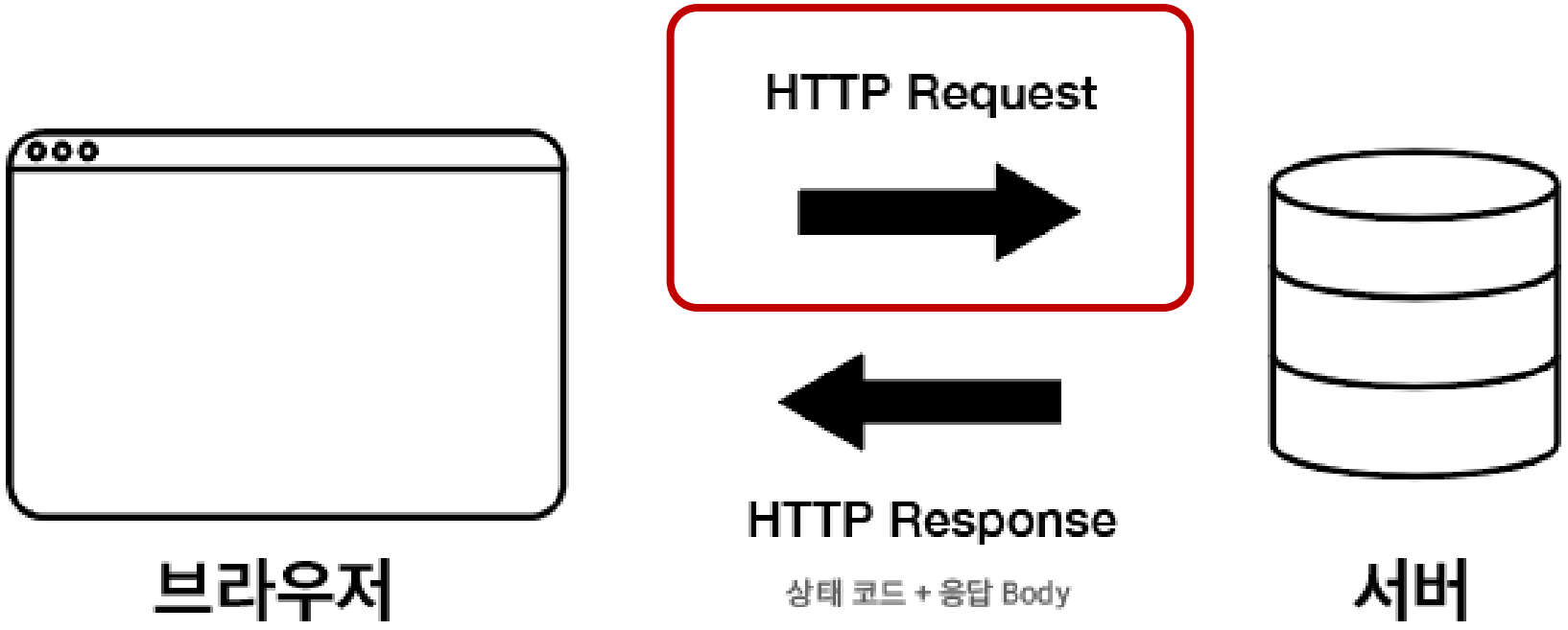
데이터의 신뢰성 있는 전송을 보장하기 위한 연결 지향형 프로토콜

HTTP



HTTP는 TCP 위에서 동작하는 프로토콜

HTTP Request



클라이언트(브라우저)가 서버에 어떤 작업을 수행해 달라고 요청하는 메시지

HTTP Request Method

GET
데이터 조회

POST
데이터 제출 및 처리

PUT
자원 생성 또는 교체

PATCH
자원 일부 수정

DELETE
자원 삭제

HTTP Request 의 구성 요소

Request Line: HTTP Request Method, 요청 경로,
HTTP 버전 등을 명시하여 요청할 작업과 대상 지정

Header: 요청에 필요한 호스트, 데이터 형식 등의 부가 정보 전달

Body: 서버에 전송할 JSON 등의 실제 데이터

HTTP Request: GET

```
GET /products HTTP/1.1
Host: shop.example.com
```

Request Line

Header

HTTP Request: POST

```
POST /users HTTP/1.1
Host: shop.example.com
Content-Type: application/json

{
  "name": "홍길동",
 "age": 20
}
```

Request Line

Header

Body

HTTP Request: PUT

```
PUT /users/1 HTTP/1.1
Host: shop.example.com
Content-Type: application/json

{
  "name": “김철수 " ,
   " age " : 25
}
```

Request Line

Header

Body

HTTP Request: PATCH

```
PATCH /users/1 HTTP/1.1
Host: shop.example.com
Content-Type: application/json

{
  "name": “이영희 "
}
```

Request Line

Header

Body

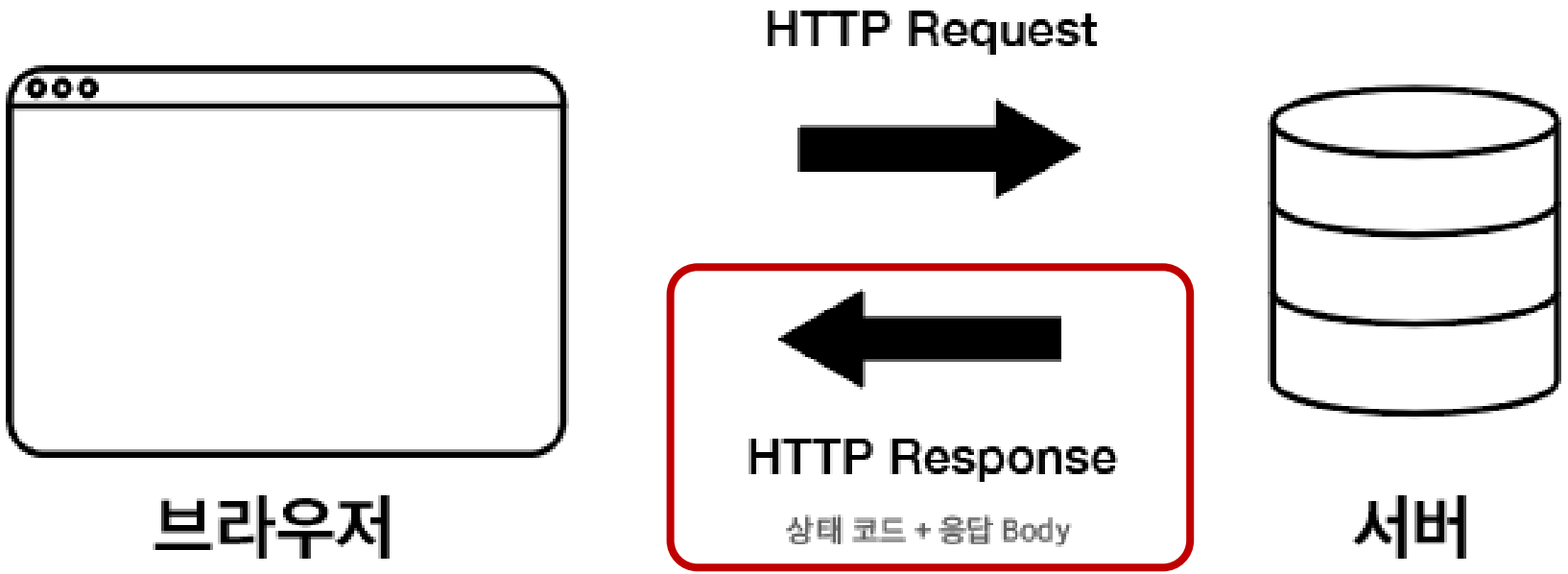
HTTP Request: DELETE

```
DELETE /users/1 HTTP/1.1
Host: shop.example.com
```

Request Line

Header

HTTP Response



클라이언트(브라우저)의 HTTP 요청에 대해 서버가 처리 결과를 반환하는 메시지

HTTP Response 의 구성 요소

Status Line: 요청의 처리 결과를 상태 코드와 메시지로 표현

Header: 응답 데이터의 형식, 크기 등 부가 정보 제공

Body: 서버가 클라이언트에 전달하는 실제 데이터

Status Code

1xx
요청 처리 중 정보 전달

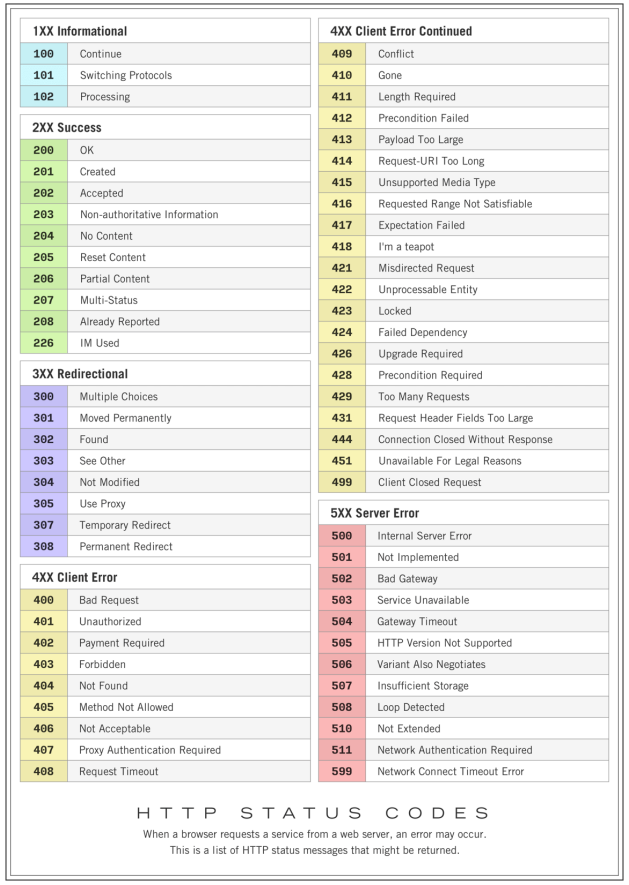
2xx
요청 성공

3xx
리디렉션 또는
캐시 사용

4xx
클라이언트 측
요청 문제

5xx
서버 측 처리 문제

Status Code (참고)



1XX Informational	
100	Continue
101	Switching Protocols
102	Processing

2XX Success	
200	OK
201	Created
202	Accepted
203	Non-authoritative Information
204	No Content
205	Reset Content
206	Partial Content
207	Multi-Status
208	Already Reported
226	IM Used

3XX Redirectional	
300	Multiple Choices
301	Moved Permanently
302	Found
303	See Other
304	Not Modified
305	Use Proxy
307	Temporary Redirect
308	Permanent Redirect

4XX Client Error	
400	Bad Request
401	Unauthorized
402	Payment Required
403	Forbidden
404	Not Found
405	Method Not Allowed
406	Not Acceptable
407	Proxy Authentication Required
408	Request Timeout

4XX Client Error Continued	
409	Conflict
410	Gone
411	Length Required
412	Precondition Failed
413	Payload Too Large
414	Request-URI Too Long
415	Unsupported Media Type
416	Requested Range Not Satisfiable
417	Expectation Failed
418	I'm a teapot
421	Misdirected Request
422	Unprocessable Entity
423	Locked
424	Failed Dependency
426	Upgrade Required
428	Precondition Required
429	Too Many Requests
431	Request Header Fields Too Large
444	Connection Closed Without Response
451	Unavailable For Legal Reasons
499	Client Closed Request

5XX Server Error	
500	Internal Server Error
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout
505	HTTP Version Not Supported
506	Variant Also Negotiates
507	Insufficient Storage
508	Loop Detected
510	Not Extended
511	Network Authentication Required
599	Network Connect Timeout Error

HTTP STATUS CODES

When a browser requests a service from a web server, an error may occur.
This is a list of HTTP status messages that might be returned.

(클릭 시 즉시 다운됨)

HTTP Response

```
GET /products HTTP/1.1

Host: shop.example.com
```

Request Line

Header



```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 1,
"name": "노트북",
 "price": 1000000
}
```

Status Line

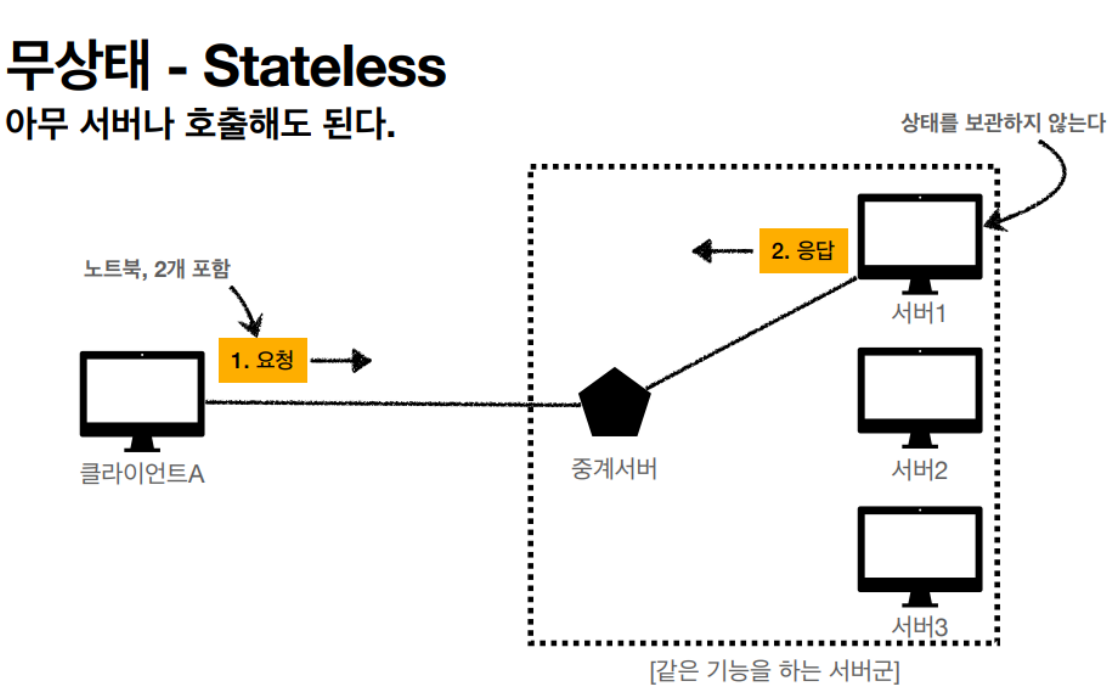
Header

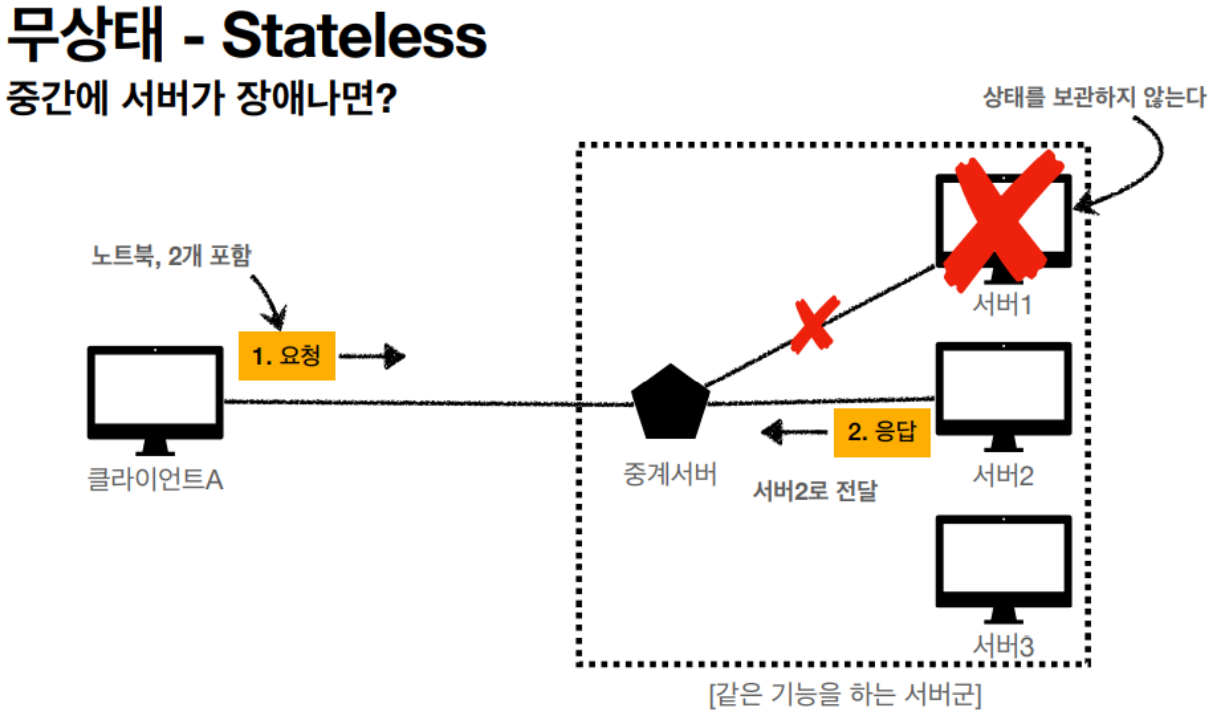
Body

03
상태와 보안

State & Security

Stateless



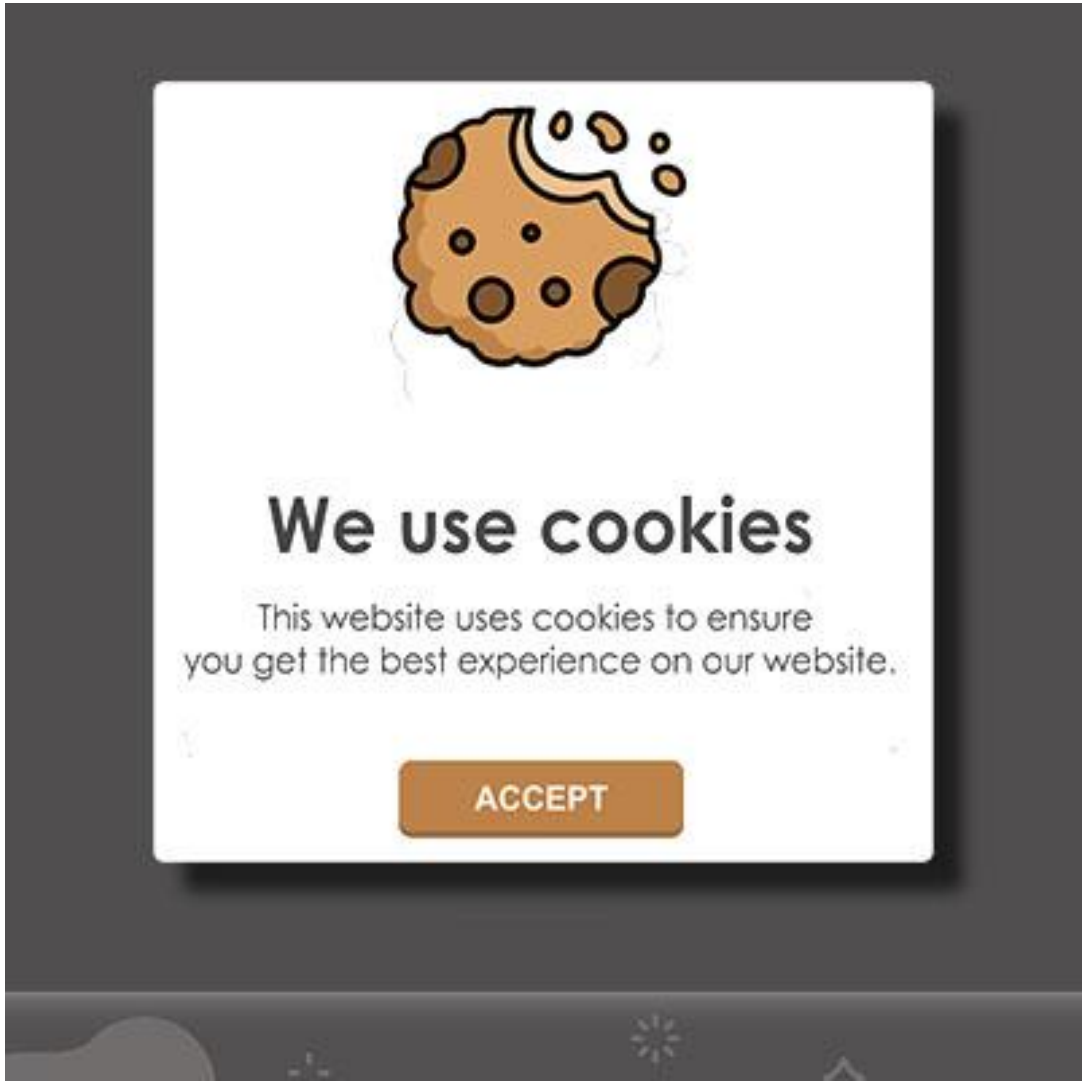


HTTP는 기본적으로 각 요청을 독립적으로 처리함

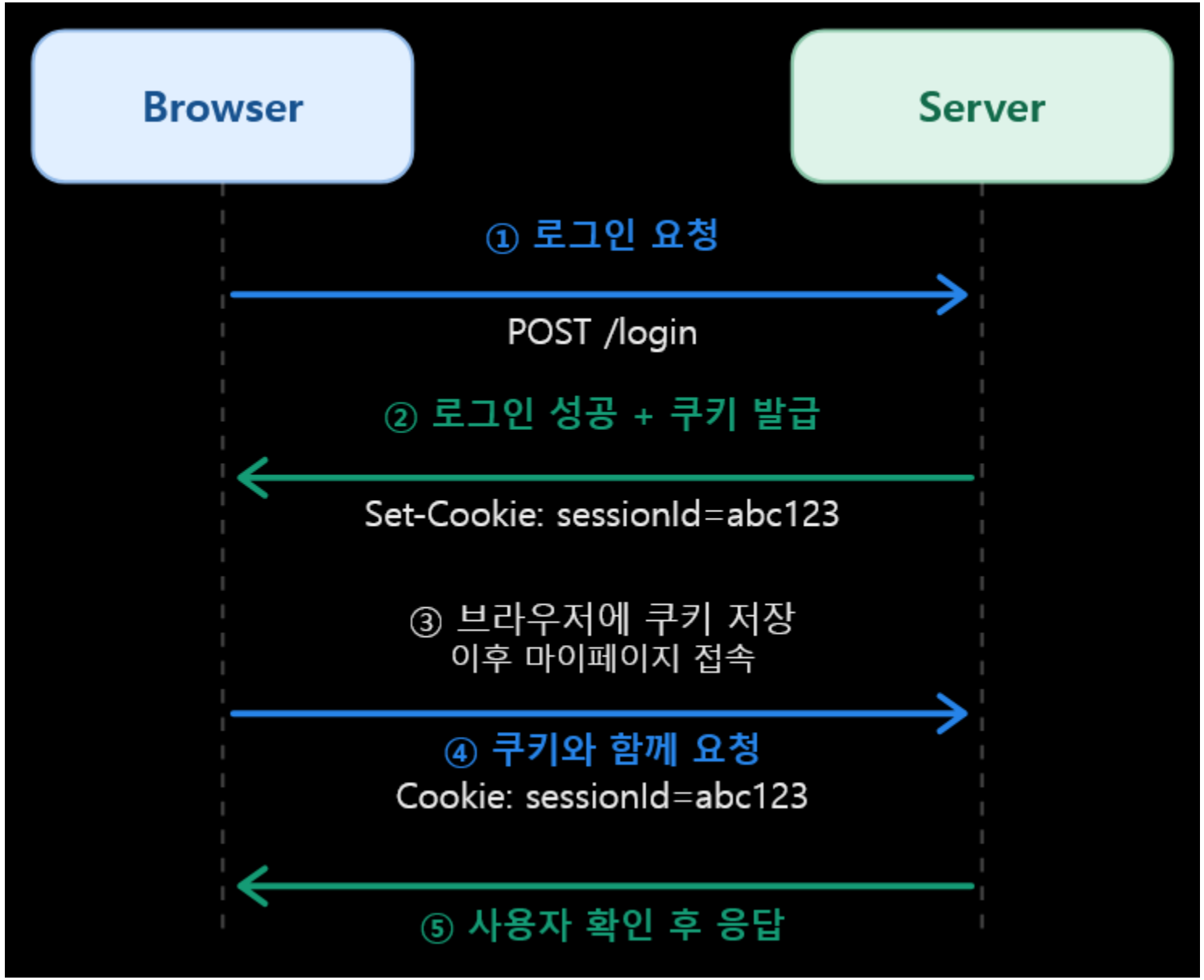
Stateless이면 방금 요청을 한 사용자가 로그인을 한 사용자인지 알 수 없음

Cookie

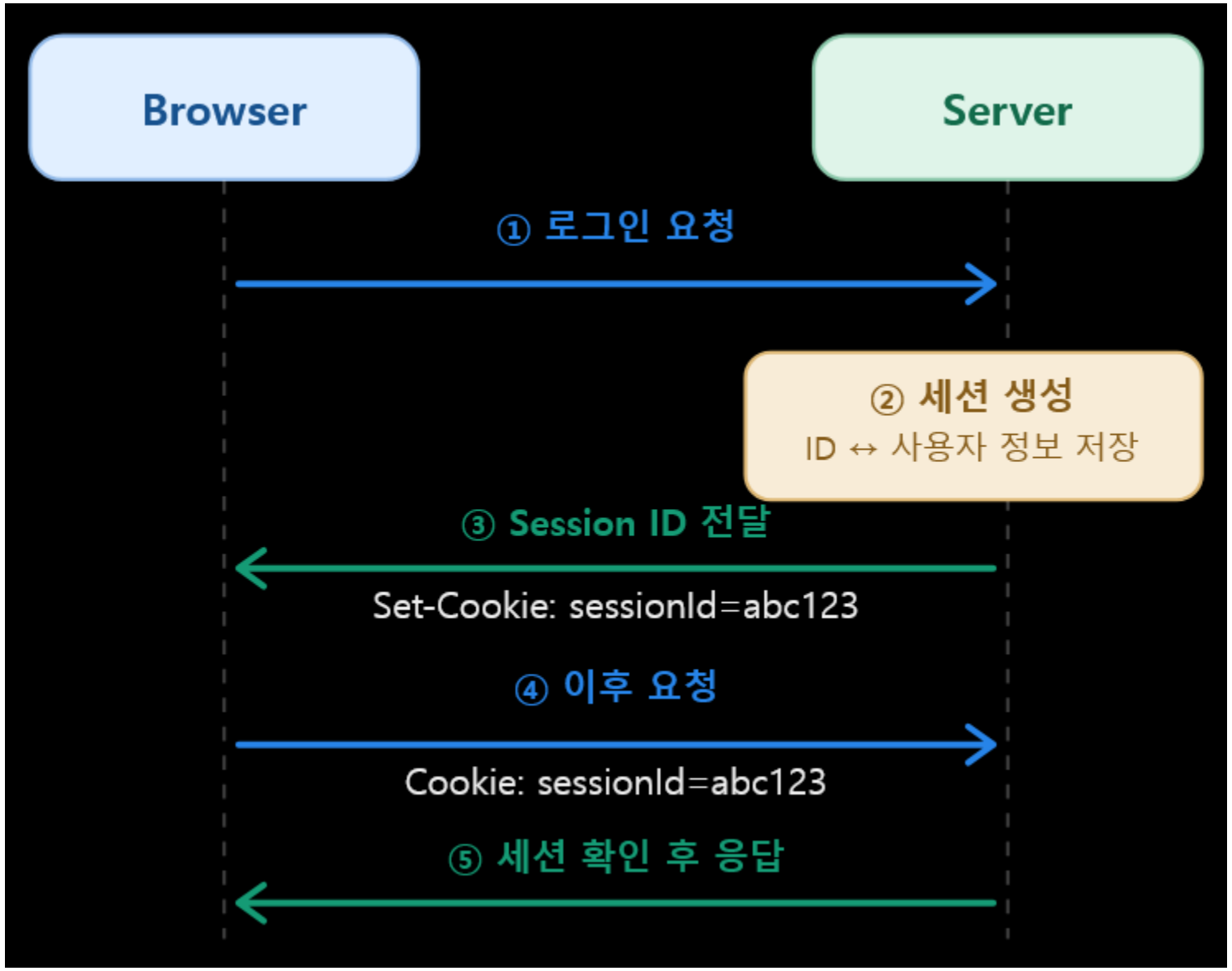
- 웹 서버가 브라우저에 저장하도록 전달하는 작은 데이터 조각
- 사용자 식별 및 상태 유지에 사용됨
- 유통기한이 있는 쿠키(Persistent Cookie), 유통기한이 없는 쿠키(Session Cookie)로 구분됨



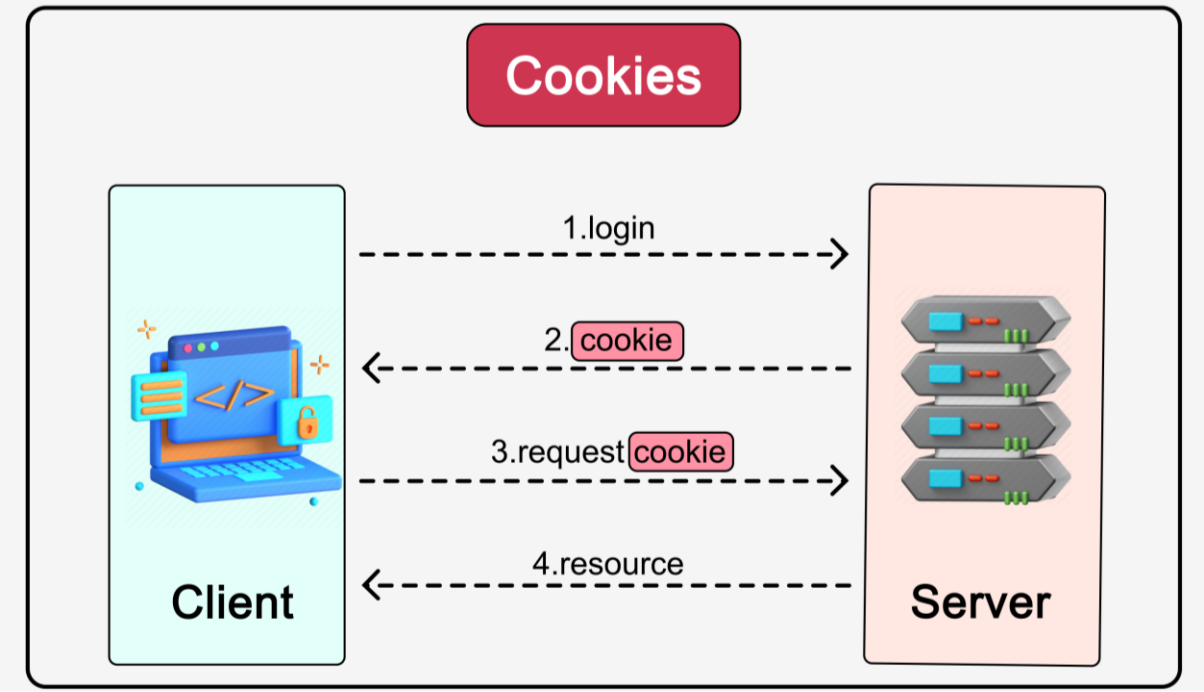
Cookie

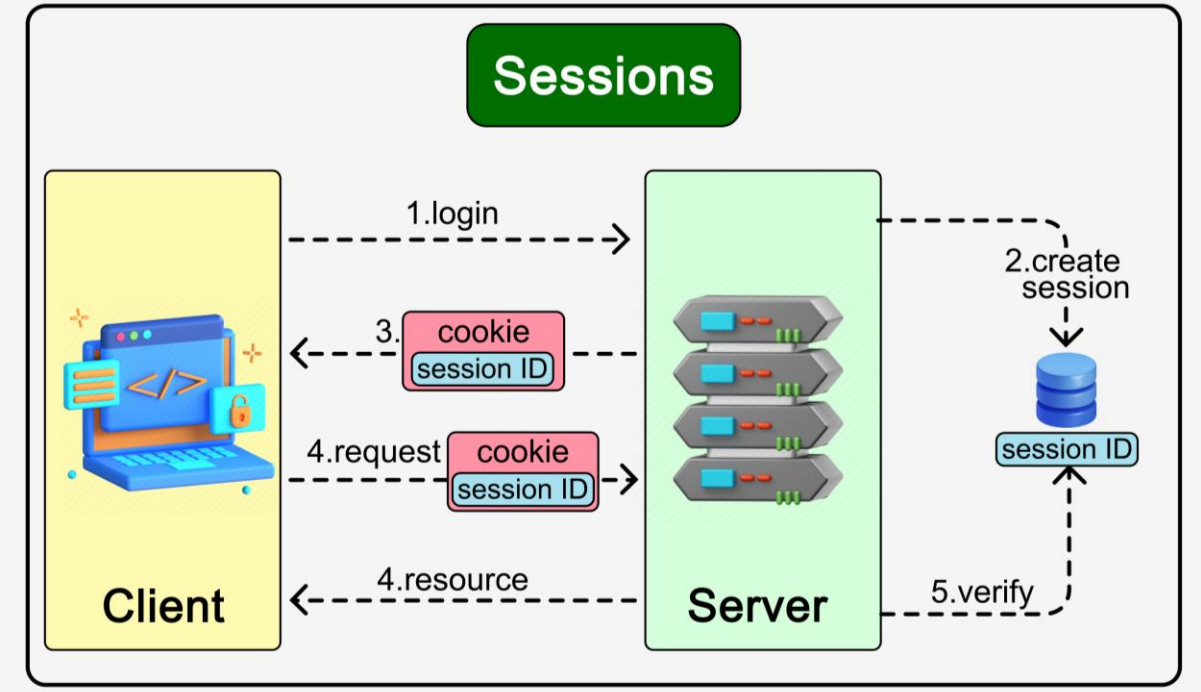

Browser
Server
① 로그인 요청
POST /login
② 로그인 성공 + 쿠키 발급
Set-Cookie: sessionId=abc123
③ 브라우저에 쿠키 저장
이후 마이페이지 접속
④ 쿠키와 함께 요청
Cookie: sessionId=abc123
⑤ 사용자 확인 후 응답

Session


Browser
Server
① 로그인 요청
② 세션 생성
ID ↔ 사용자 정보 저장
③ Session ID 전달
Set-Cookie: sessionId=abc123
④ 이후 요청
Cookie: sessionId=abc123
⑤ 세션 확인 후 응답

Session





Cookie: 데이터를 클라이언트에 저장하여 사용자 상태 유지

Session: 상태 데이터를 서버에 저장하고 Session ID로 사용자 식별

HTTP의 문제점 (보안 측면에서)

```
POST /login HTTP/1.1
Host: example.com

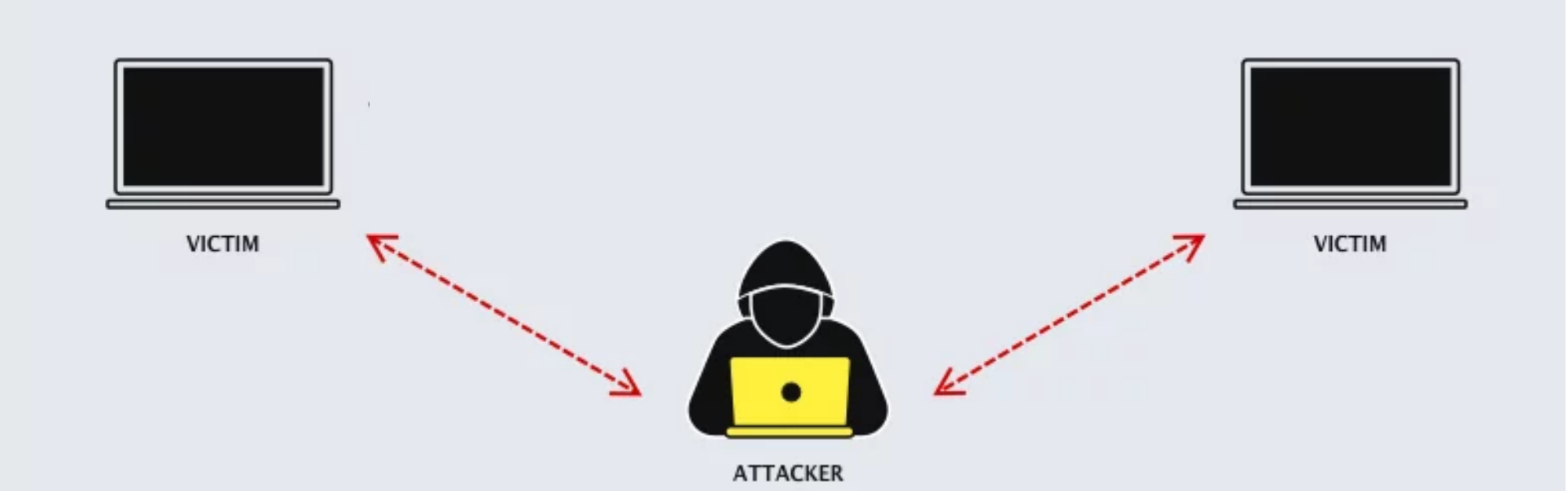
id=user1&password=1234
```

기밀성 부족

HTTP는 데이터를 암호화하지 않고 평문으로 전송

공격자가 통신을 가로채면 아이디, 비밀번호같은 민감한 정보를 읽을 수 있음

HTTP의 문제점 (보안 측면에서)



무결성 부족

HTTP 자체에는 전송 중 데이터가 변경되었는지 알 수 없음

공격자가 데이터를 가로채서 내용을 변경하는 것이 가능

HTTP의 문제점 (보안 측면에서)



https://**guthib**.com

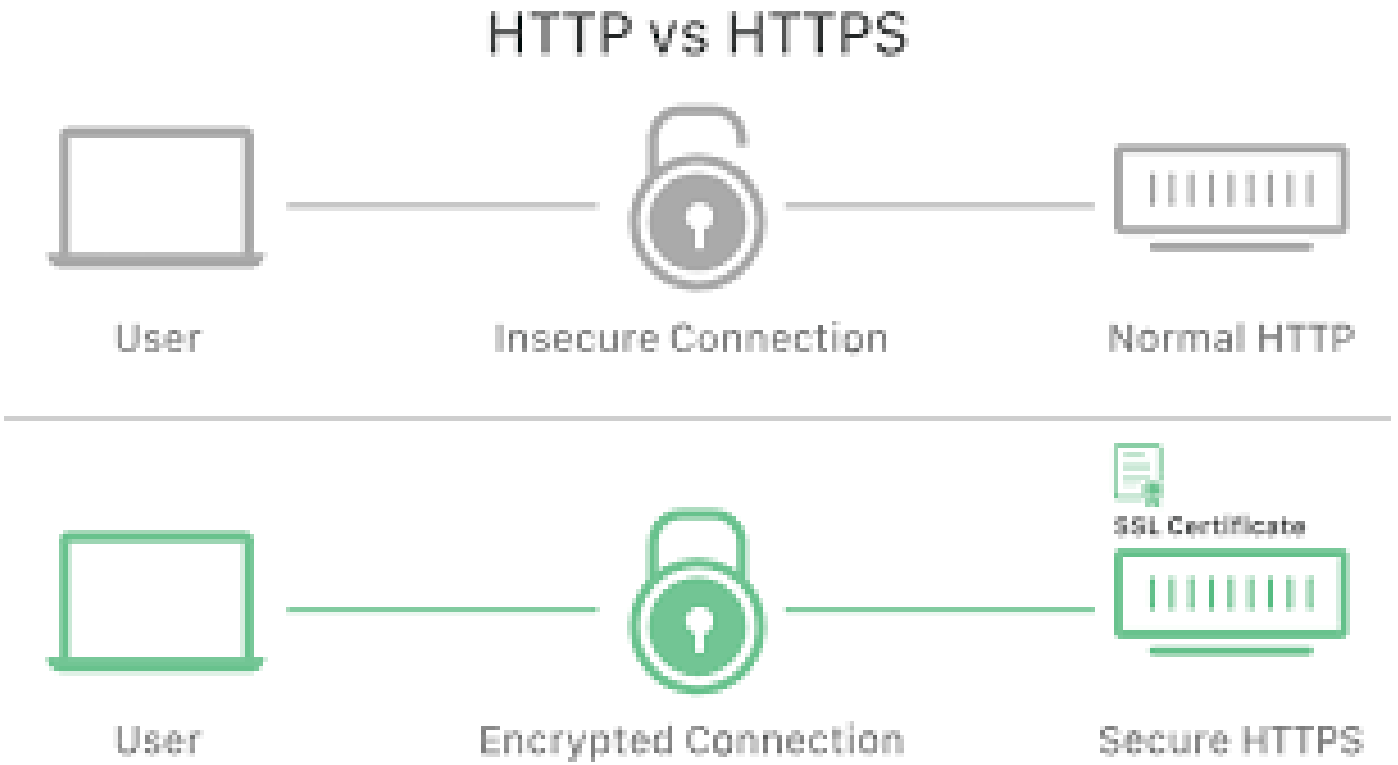
You spelled it wrong.

인증 부족

HTTP 자체는 접속한 서버가 신뢰할 수 있는 서버인지 알 수 없음

공격자가 가짜 서버로 유도하거나 통신을 가로채는 공격 가능

해결책: 보안 프로토콜 추가



HTTPS

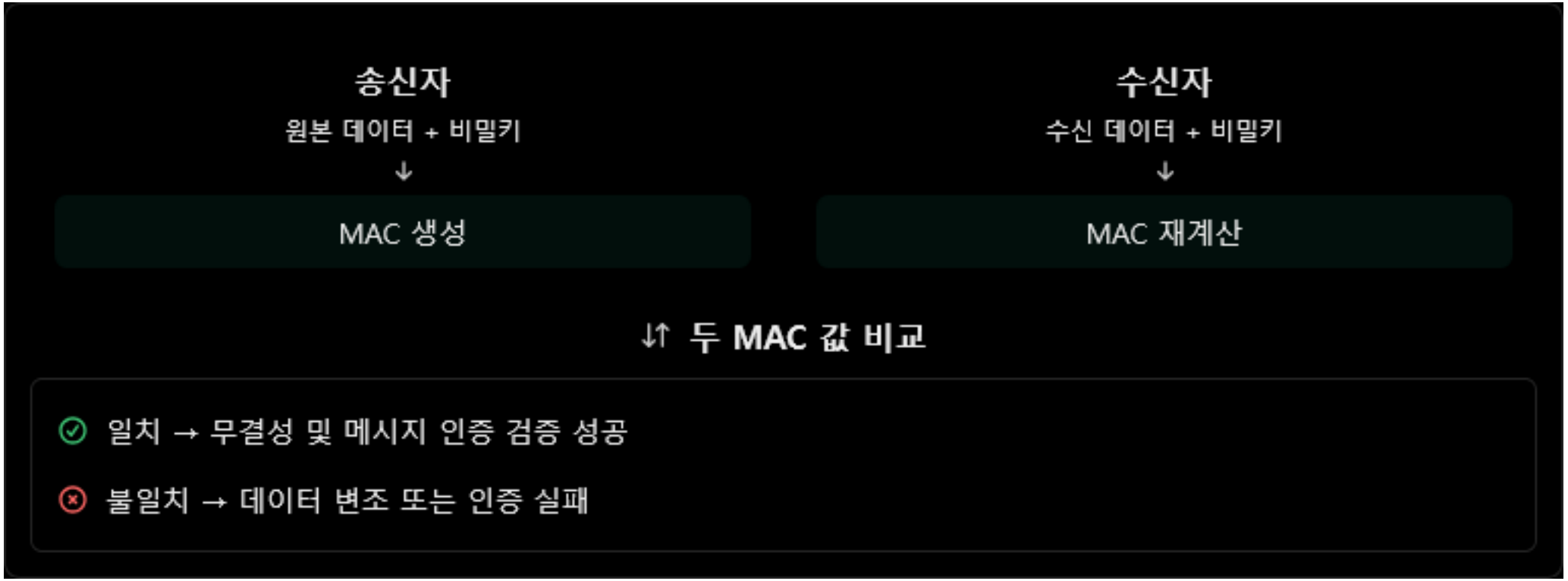
HTTP + TLS(Transport Layer Security)

Encryption: 기밀성 해결



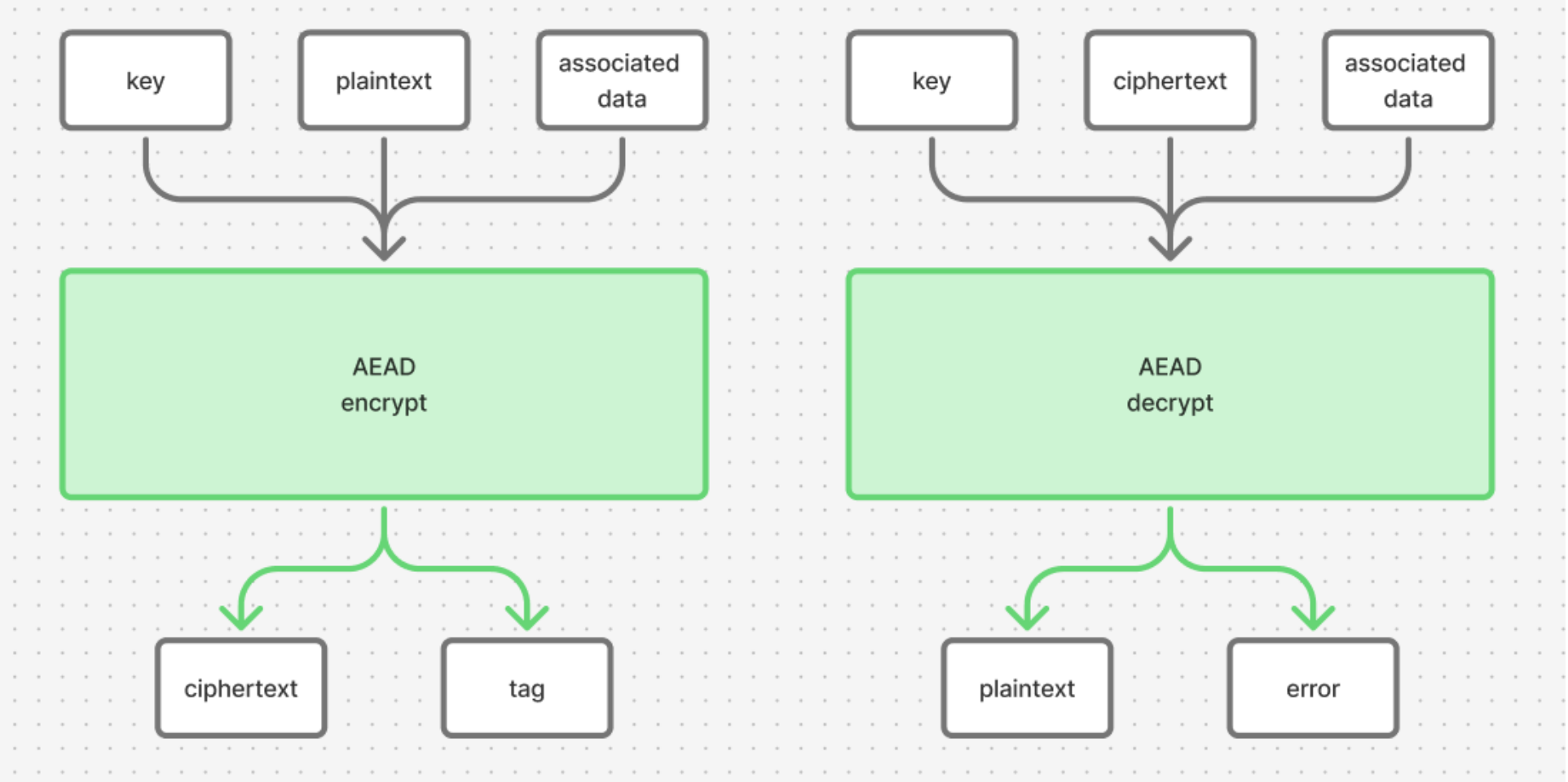
- 암호화 및 복호화 시에는 키를 사용함
- 암호화와 복호화에 같은 키를 사용하면 대칭 키 암호화(Symmetric Encryption)라고 부름
- 비대칭 키 암호화는 공개 키(Public Key)와 개인 키(Private Key)라는 서로 다른 키를 이용함
  공개 키는 누구나 접근 가능하고, 공개 키로 암호화된 데이터를
  해당 공개 키와 쌍을 이루는 개인 키로 복호화할 수 있음

MAC: Message Authentication Code



송신자가 원본 데이터와 비밀 키로 MAC 값을 생성하고,
수신자가 생성한 MAC 값과 비교함
MAC 값이 일치하다는 것은 무결성을 만족한다는 뜻

AEAD: Authenticated Encryption with Associated Data

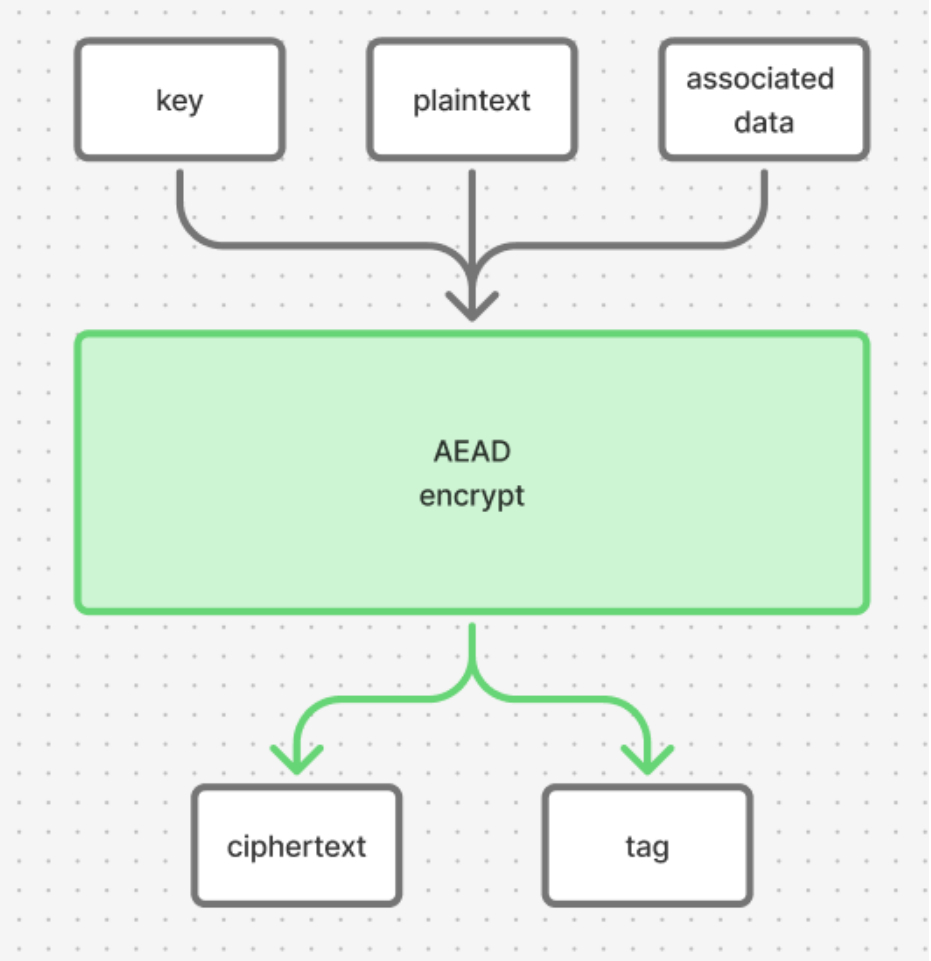


AEAD: Authenticated Encryption with Associated Data

- AEAD의 암호화 과정에서는 키, 원본 데이터, 무결성을 보호할 추가 데이터 이렇게 세 가지 입력이 들어감

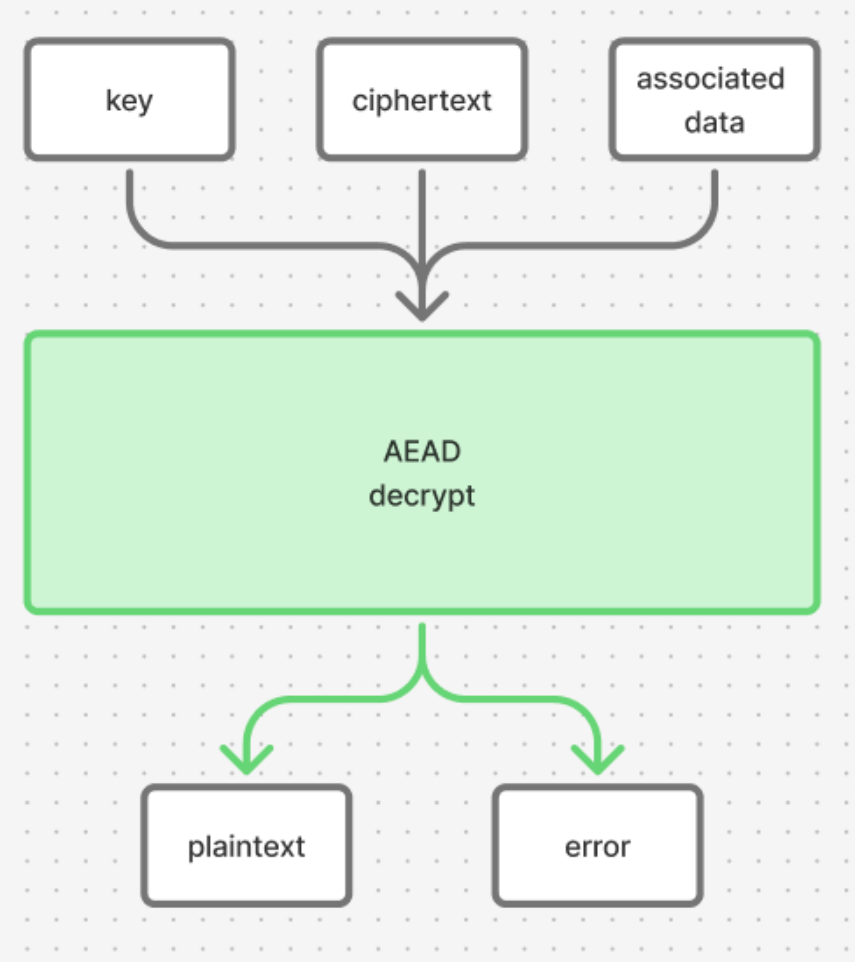
- AEAD Encryption 알고리즘에 넣으면 원본 데이터의 암호화한 데이터와 데이터가 변조되었는지 검증하기 위한 값인 Tag(인증 태그)가 출력됨

- 인증 태그는 암호문과 키 등의 정보로 계산된 값



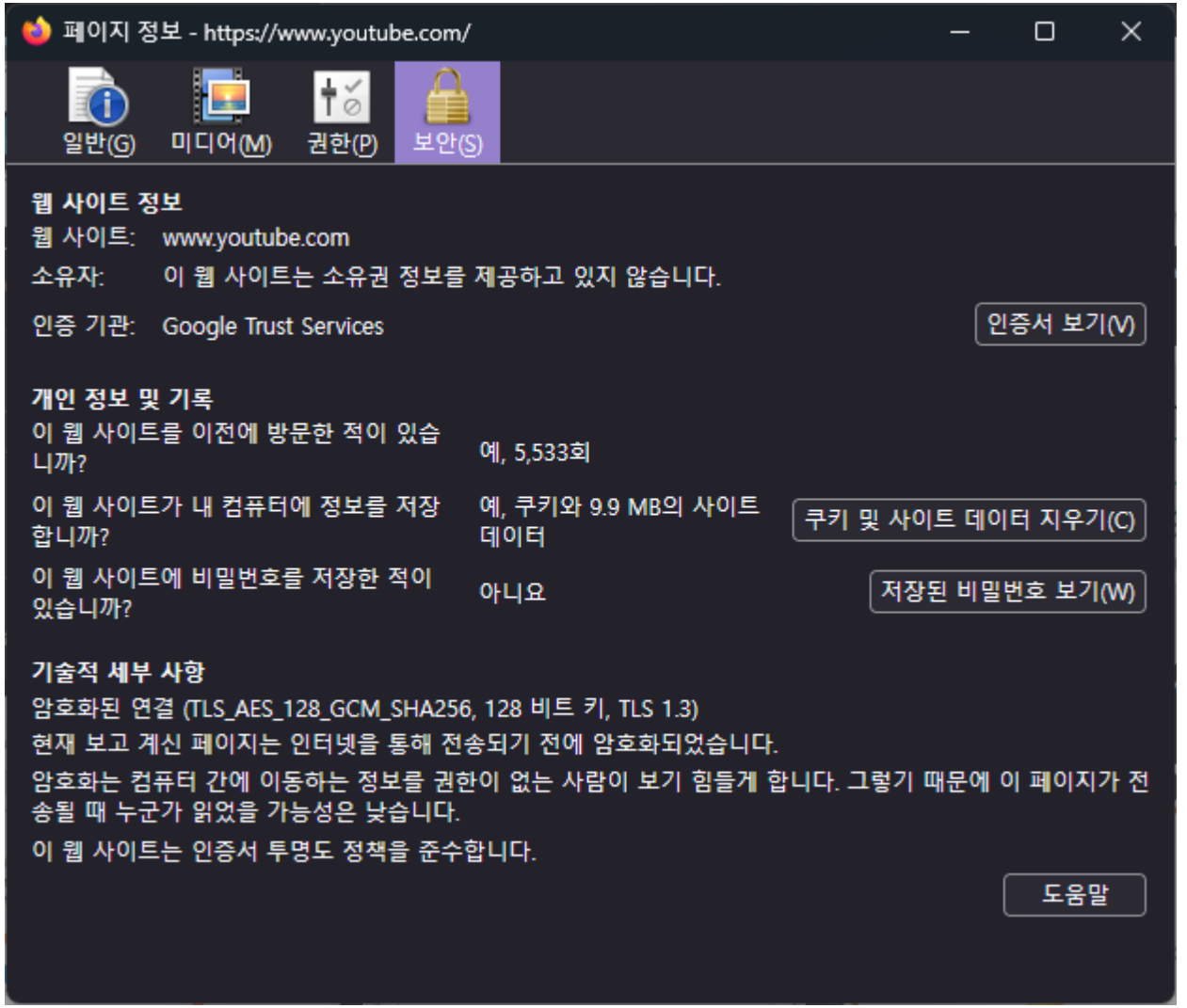
AEAD: Authenticated Encryption with Associated Data

- AEAD의 복호화 과정에서는 암호화 시 사용한 비밀 키, 암호화된 데이터, 암호화할 때 사용한 추가 데이터, 인증 태그가 입력됨
- 외부에서 읽어도 되지만 변경은 안되는 값을 보호하기 위해 Associated Data 사용
- 복호화 알고리즘은 수신한 데이터로 인증 태그 계산, 수신한 태그와 대조해서 데이터가 중간에 변조되었는지 검증함
- 암호화 + 무결성 검증을 동시에 할 수 있는 알고리즘

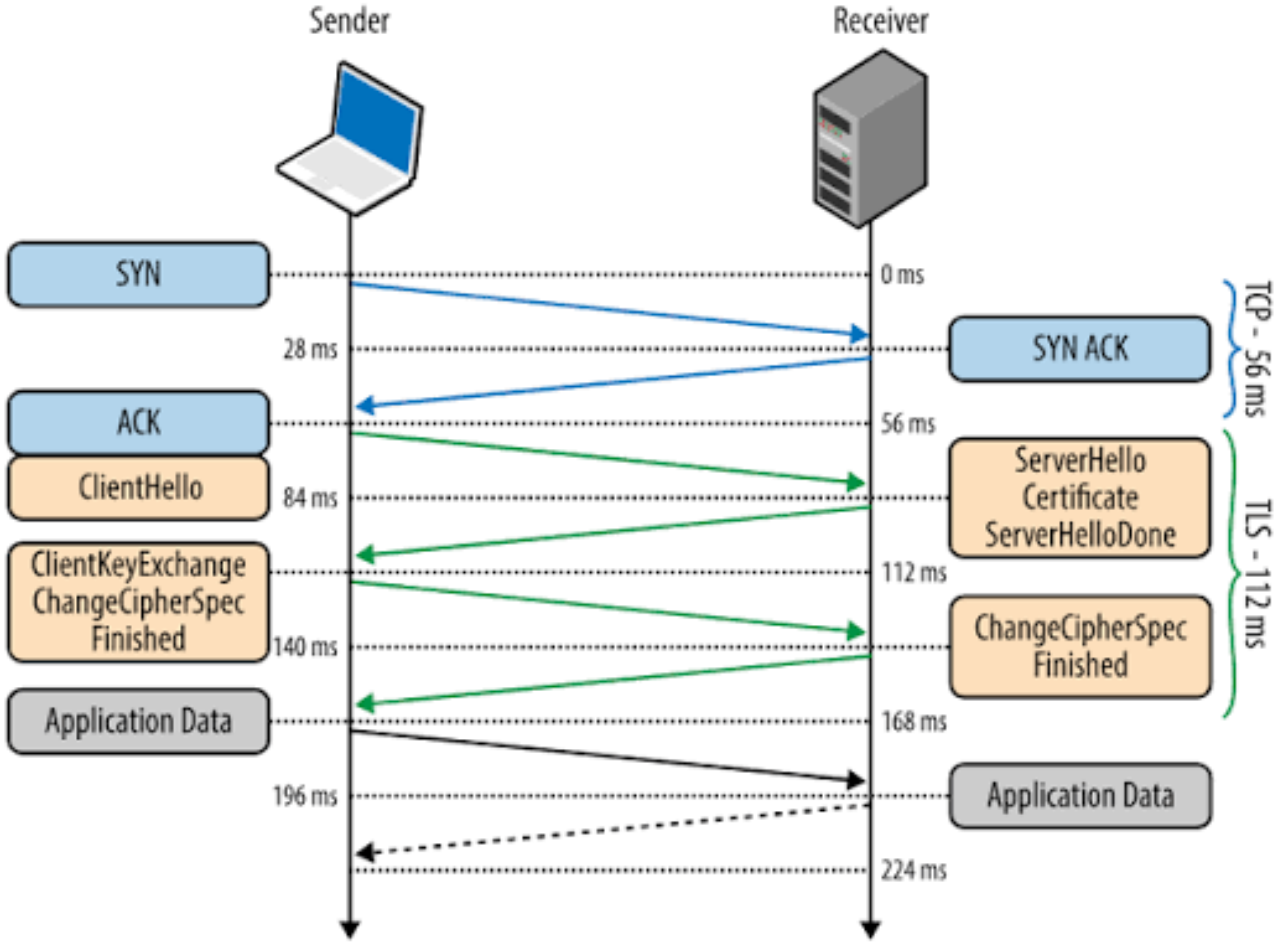


Certificate: 인증 해결

- Certificate Authority가 신뢰할 수 있는 사이트에 인증서 발급
  (이미지에서는 Google Trust Services)
- 서버가 인증서 전달
- 브라우저가 신뢰할만한 인증서인지, 유효기간이 지나진 않았는지 등을 검증함
- 인증서 검증이 완료되면 브라우저와 서버는 데이터 교환 시작

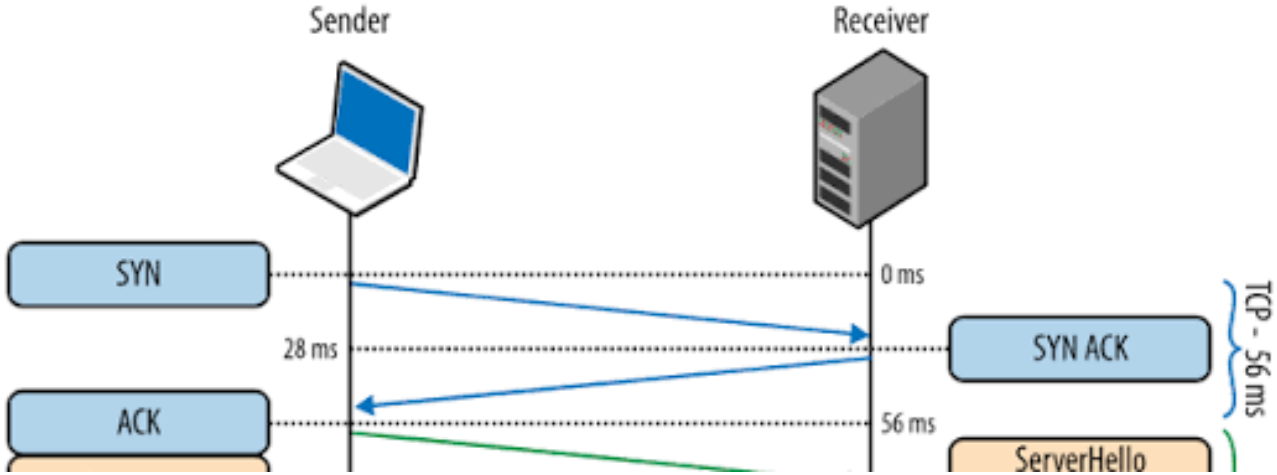


TLS: Transport Layer Security



Encryption + AEAD + Certificate 를 모두 제공하는 보안 프로토콜

TLS: Transport Layer Security

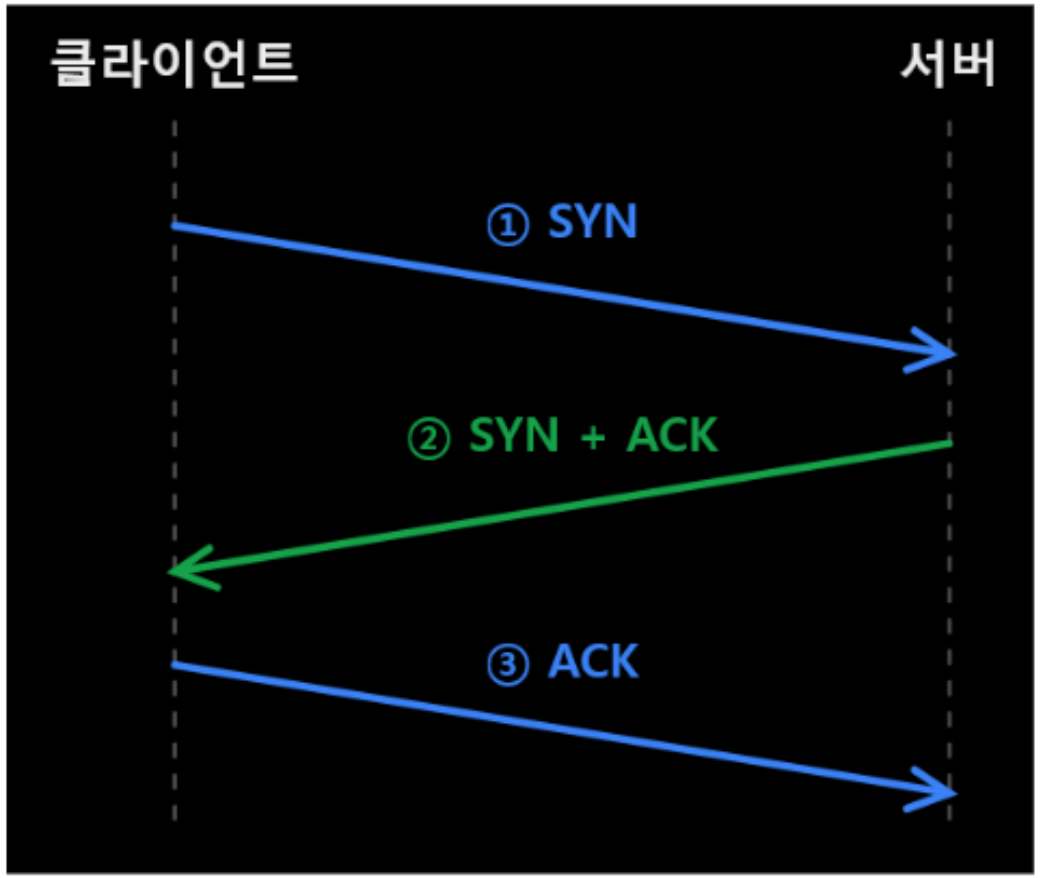


TCP Handshake

TLS 시작 전 먼저 TCP 연결을 수립하는 과정
TCP 3-Way Handshake 진행

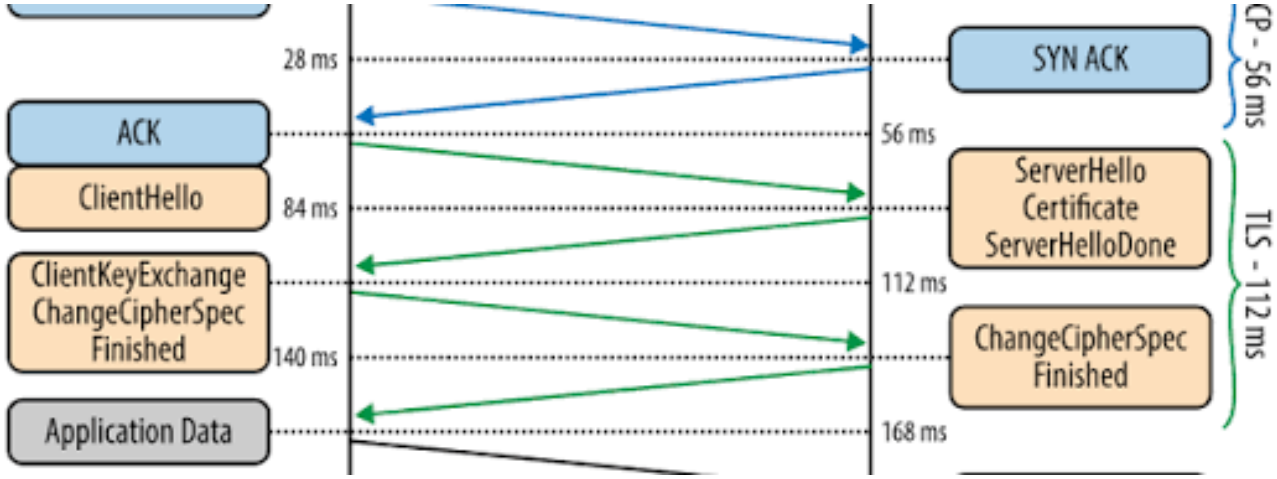
3-Way Handshake

- TCP는 데이터를 전송하기 전 클라이언트와 서버 사이에 먼저 연결을 수립하는데, 그 연결을 수립하는 과정임
- 클라이언트와 서버 사이 3번의 메시지를 주고받는 과정으로 구성됨
- 클라이언트가 서버에 SYN(Synchronize의 약자) 신호를 보내고, 서버와 클라이언트 사이 ACK를 교환해서 통신 수립



CS 10주차 TCP/IP 46페이지

TLS: Transport Layer Security



TLS Handshake

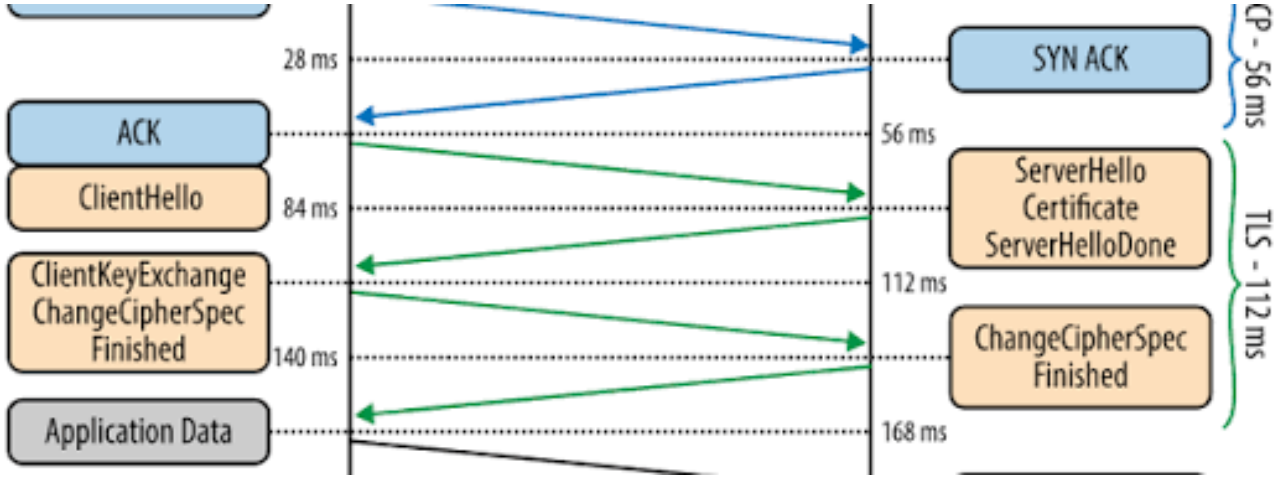
1. ClientHello

클라이언트가 서버에 TLS 통신을 시작하자고 요청

2. ServerHello + Certificate + ServerHelloDone

서버에서 인증서 전달, 서버가 사용할 암호화 방식 선택

TLS: Transport Layer Security



TLS Handshake

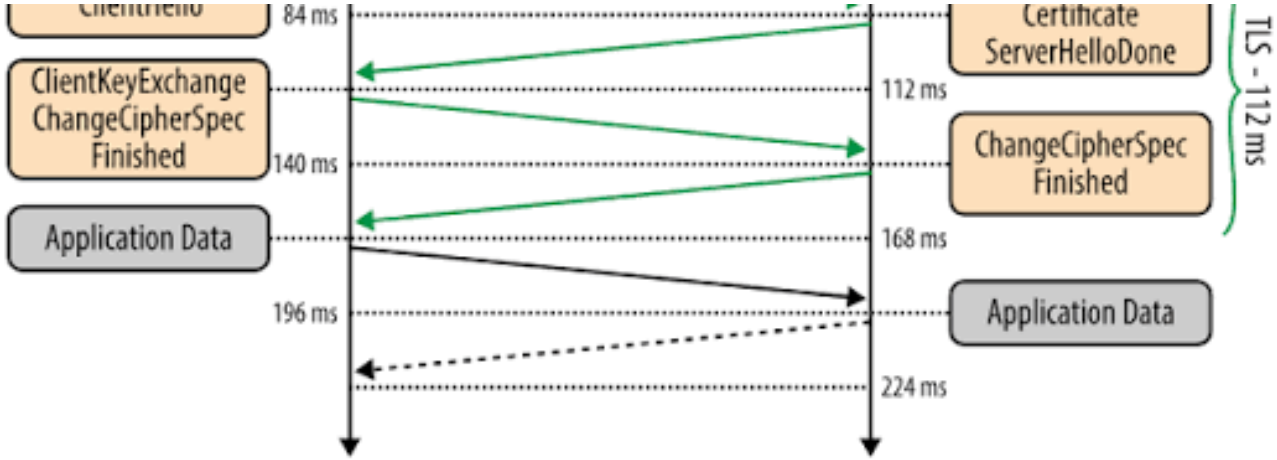
3. ClientKeyExchange + ChangeCipherSpec + Finished

암호화 통신에 필요한 키 설정, 협상한 암호화 설정을 사용하겠다고 알림

4. ChangeCipherSpec + Finished

서버도 암호화 설정 적용, Finished 메시지 전달

TLS: Transport Layer Security



Application Data

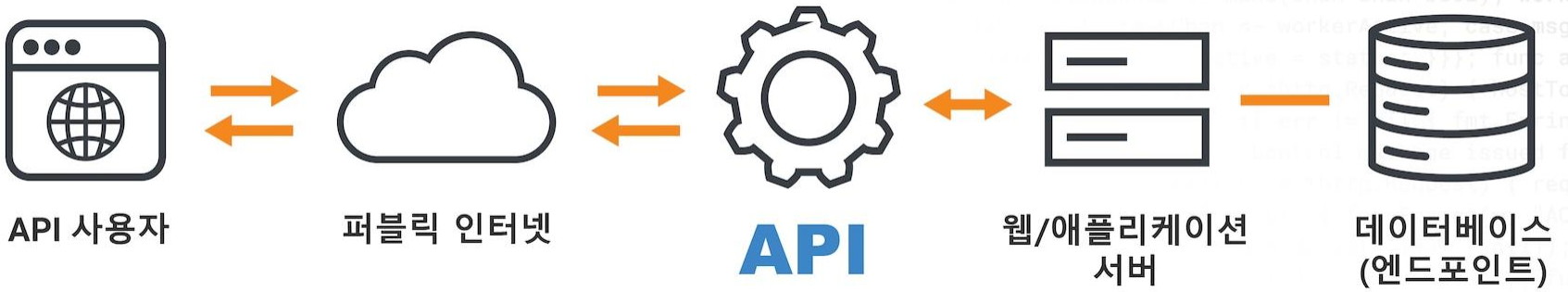
TCP 연결, TLS Handshake가 완료되면 실제 HTTP 데이터 전송

이 때 데이터를 전송하며 Encryption, MAC/AEAD가 사용됨

04
API

Application Programming Interface

API: Application Programming Interface



한 프로그램이 다른 프로그램의 기능을

사용할 수 있도록 제공하는 사전에 약속된 연결 방식

REST: Representational State Transfer



웹 API를 설계할 때 주로 쓰이는 아키텍처 스타일

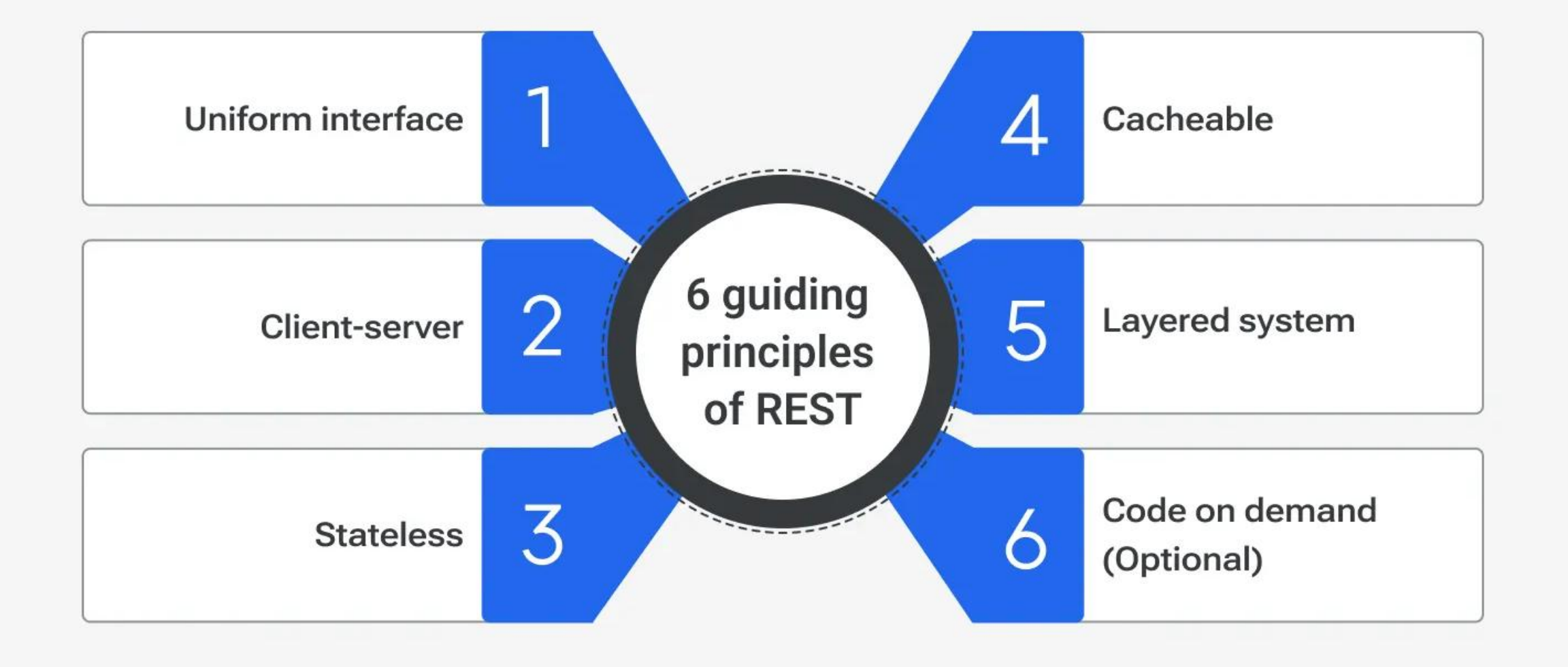
REST의 개념

Resource: 사이트의 상품, 사용자, 게시글 등 다루려는 대상

Representation: 자원의 상태를 나타내는 데이터 (JSON, XML같은)

HTTP Method: GET, POST, PUT, DELETE 같은 수행할 작업들

REST의 원칙


Uniform interface
1
Client-server
2
Stateless
3
6 guiding principles of REST
4
Cacheable
5
Layered system
6
Code on demand (Optional)

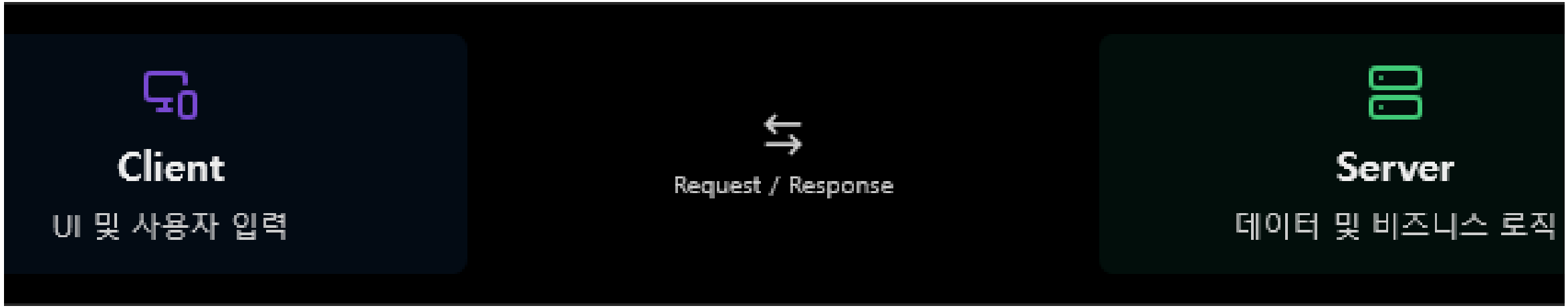
1. Uniform Interface

HTTP Method	URL	역할
GET	`/products/10`	상품 조회
PUT	`/products/10`	상품 전체 교체
DELETE	`/products/10`	상품 삭제

같은 자원에 대해서는 일관된 규칙을 통해

자원을 식별하고 관리해야 함

2. Client - Server

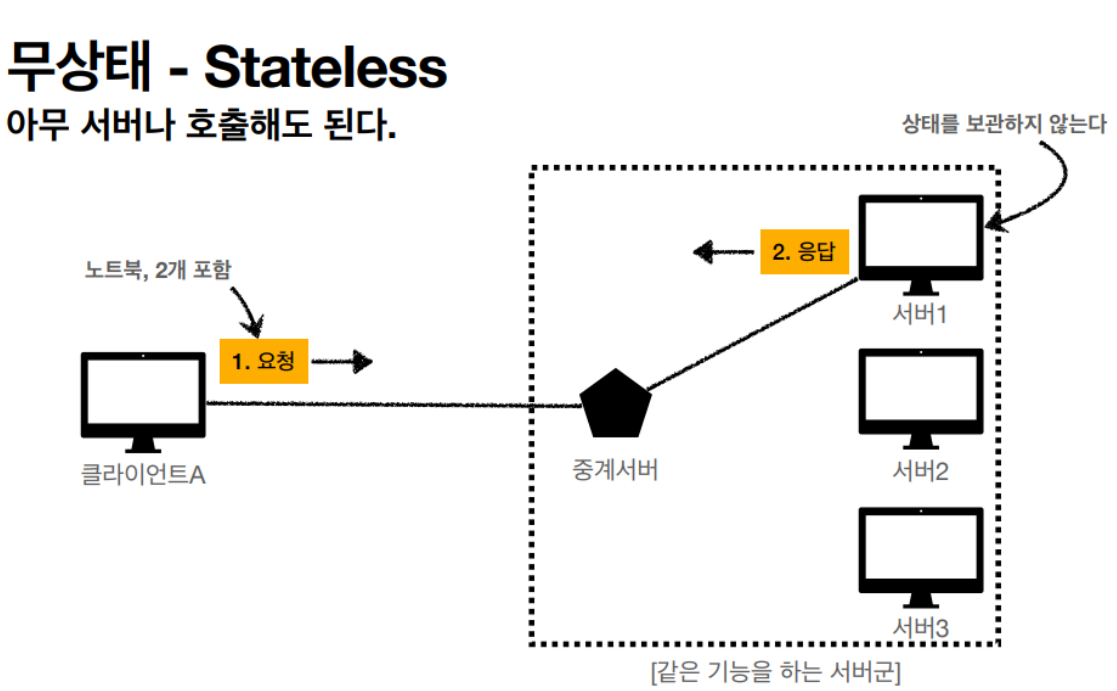


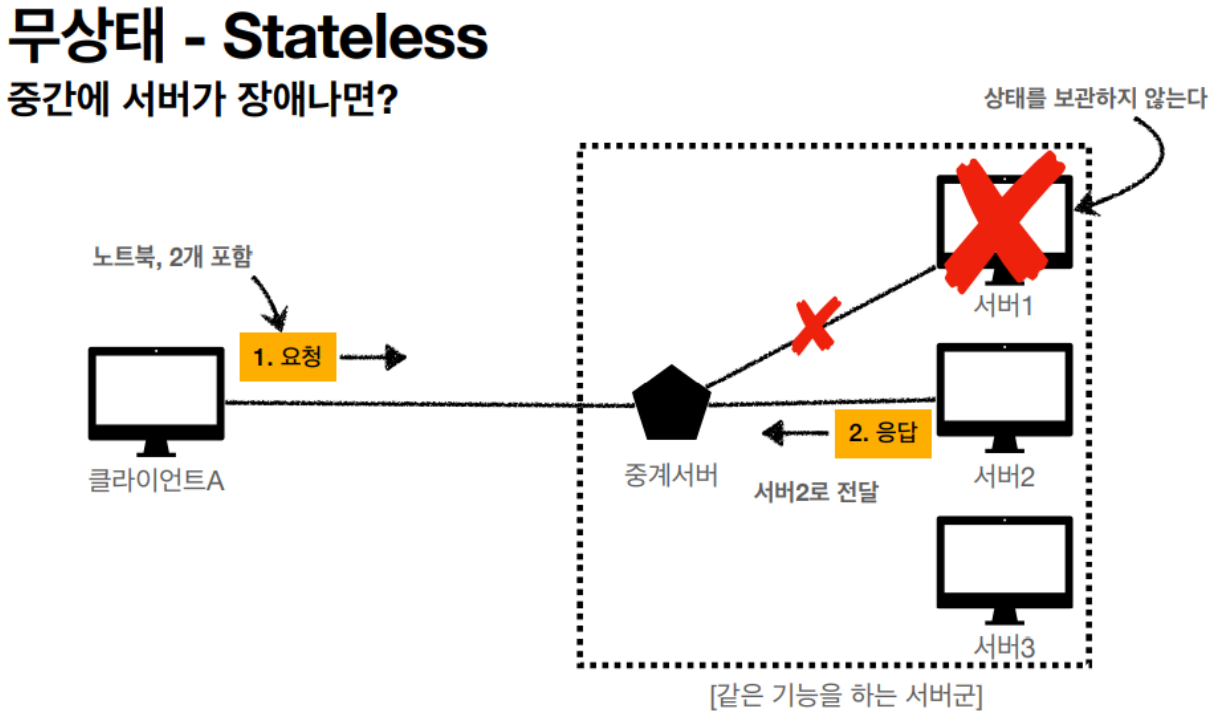
클라이언트와 서버의 역할을 분리하여

각각 독립적으로 개발 및 관리해야 함

(예시: 쇼핑몰의 디자인 변경은 상품 조회 기능에 영향을 끼치면 안됨)

3. Stateless





서버는 이전 요청 정보를 기억하지 않고,

각 요청을 독립적으로 처리해야 함

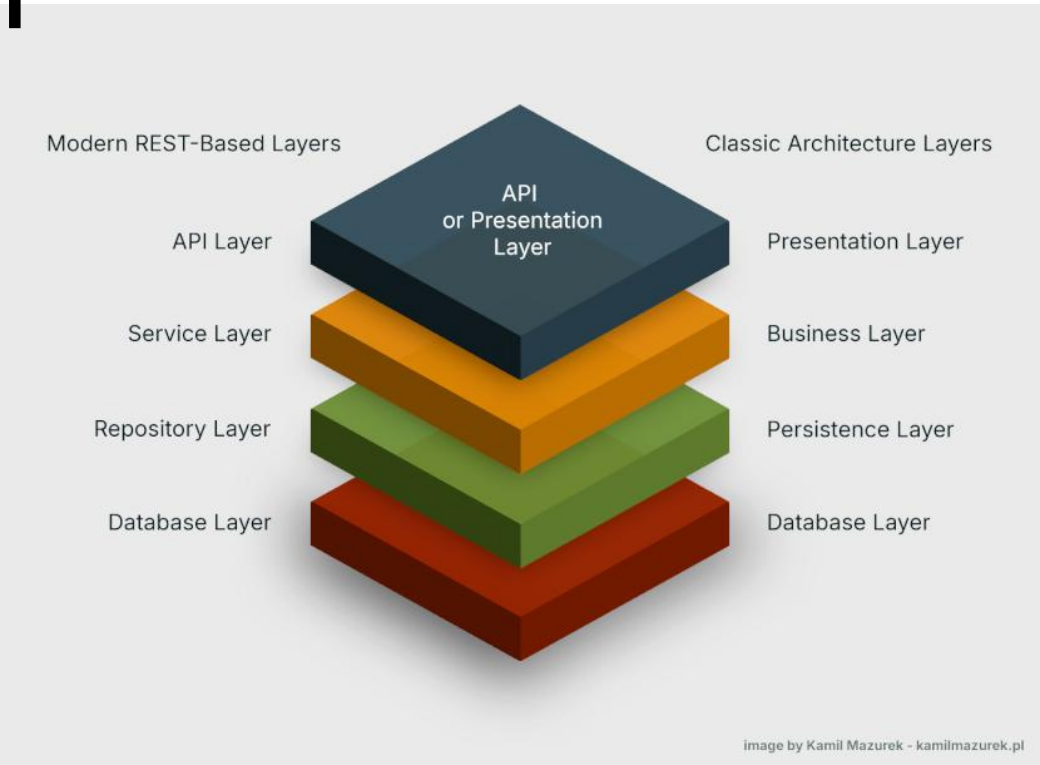
4. Cacheable



서버의 응답이 캐시 가능한 데이터인지 명시하여

불필요한 요청과 데이터 전송을 최소화해야 함

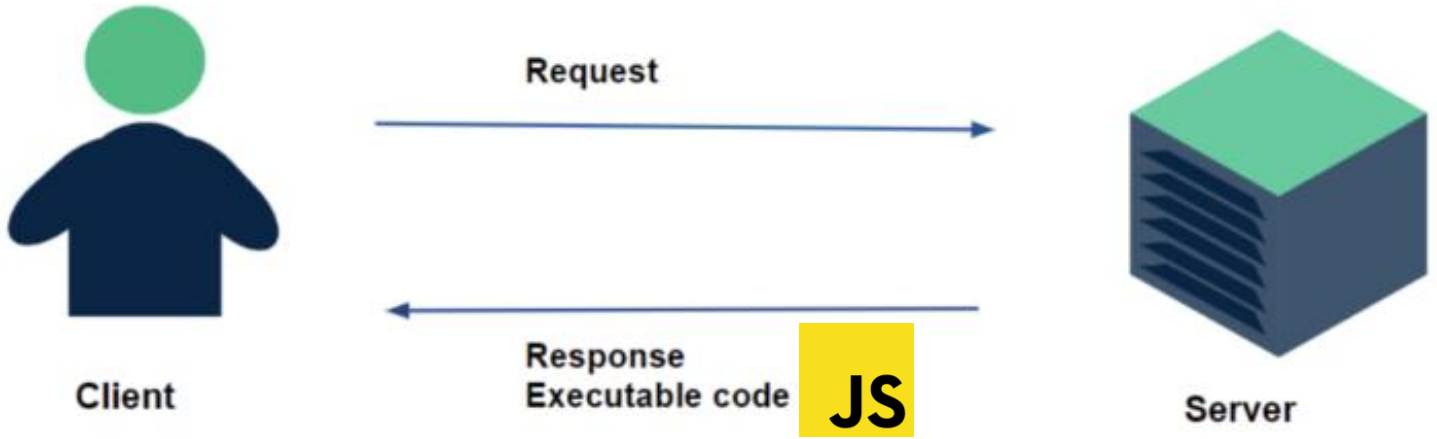
5. Layered System



클라이언트와 서버 사이에 여러 계층을 두어

시스템 확장 및 유지보수, 부하 분산에 유리하게 설계해야 함

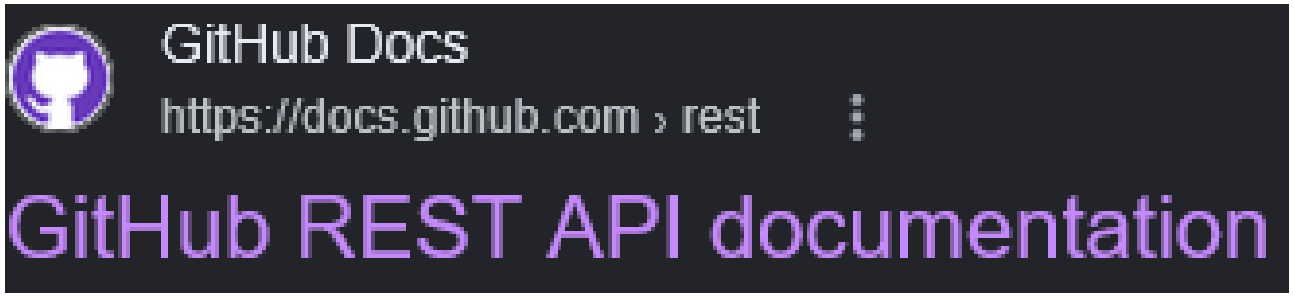
6. Code on Demand (Optional)



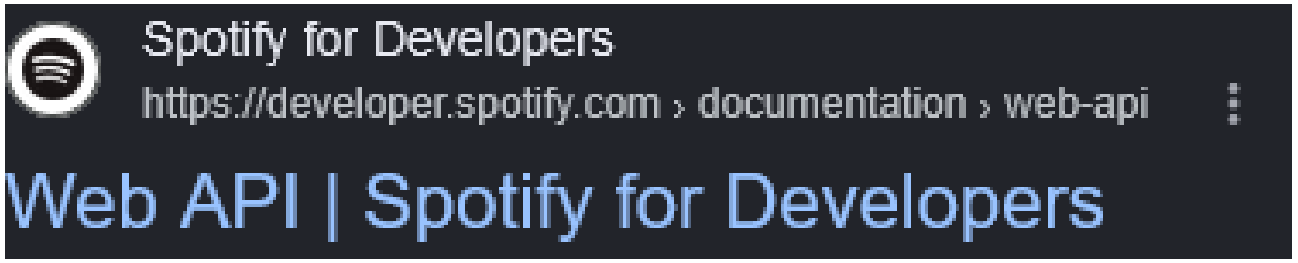
서버가 클라이언트에 실행 가능한 코드를 전달해

클라이언트의 기능을 확장할 수 있게 해야 함 (선택)

RESTful









REST의 아키텍처 제약 조건을 따르도록

설계된 시스템이나 API를 RESTful이라 함